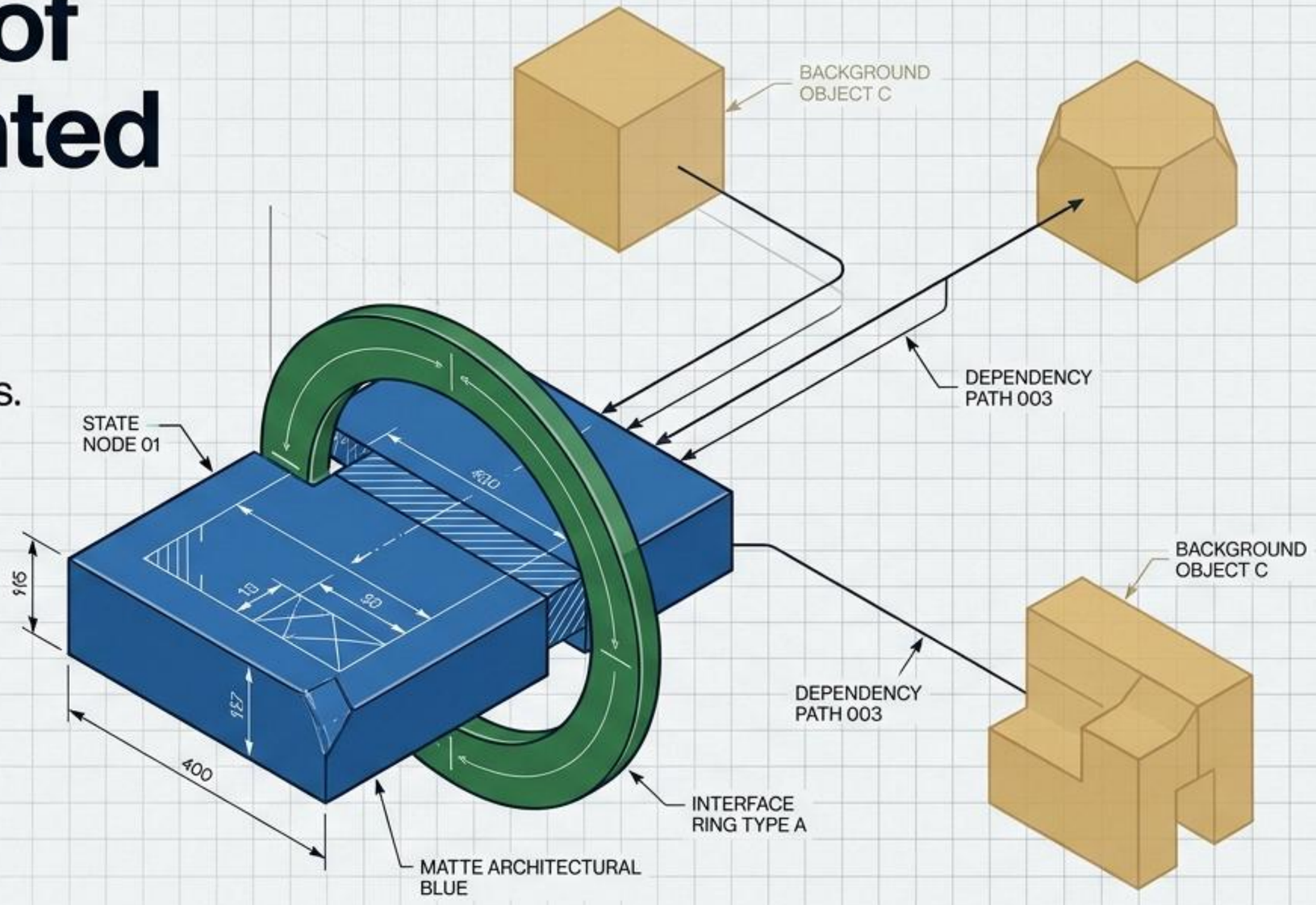
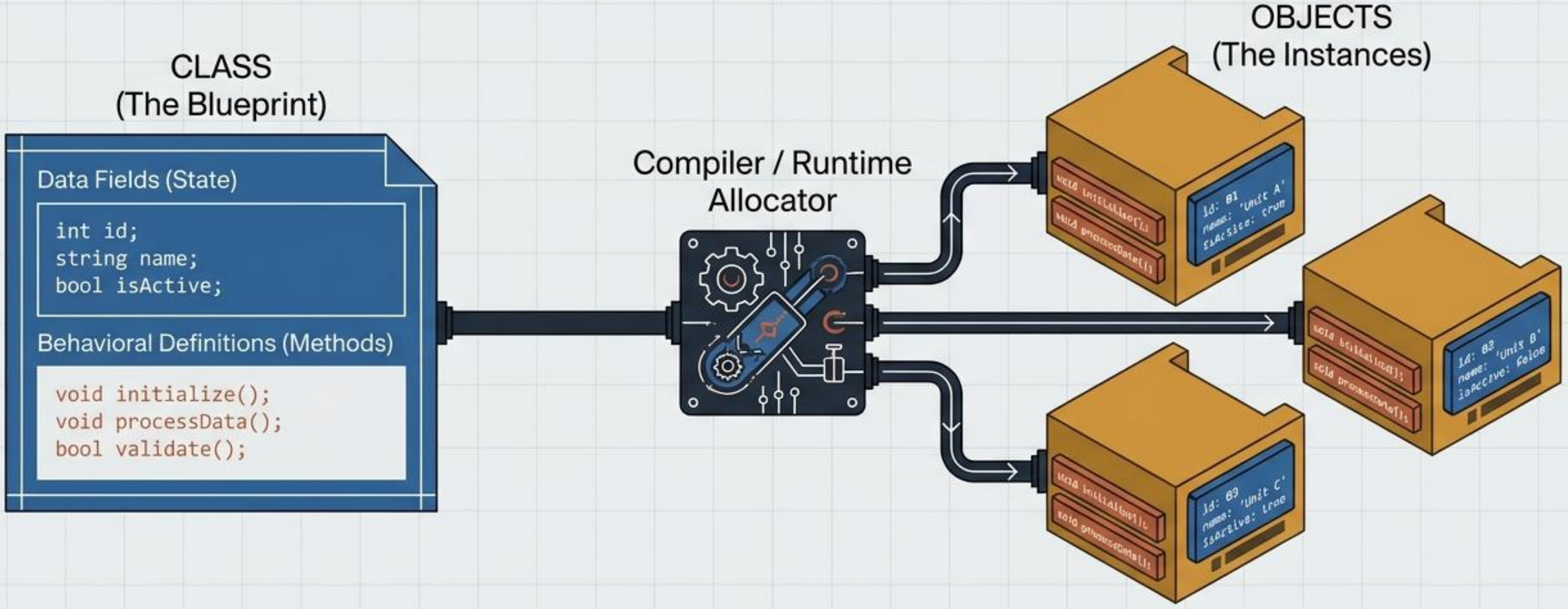
The Physics of Object-Oriented Architecture

From fundamental primitives to the Gang of Four design principles.

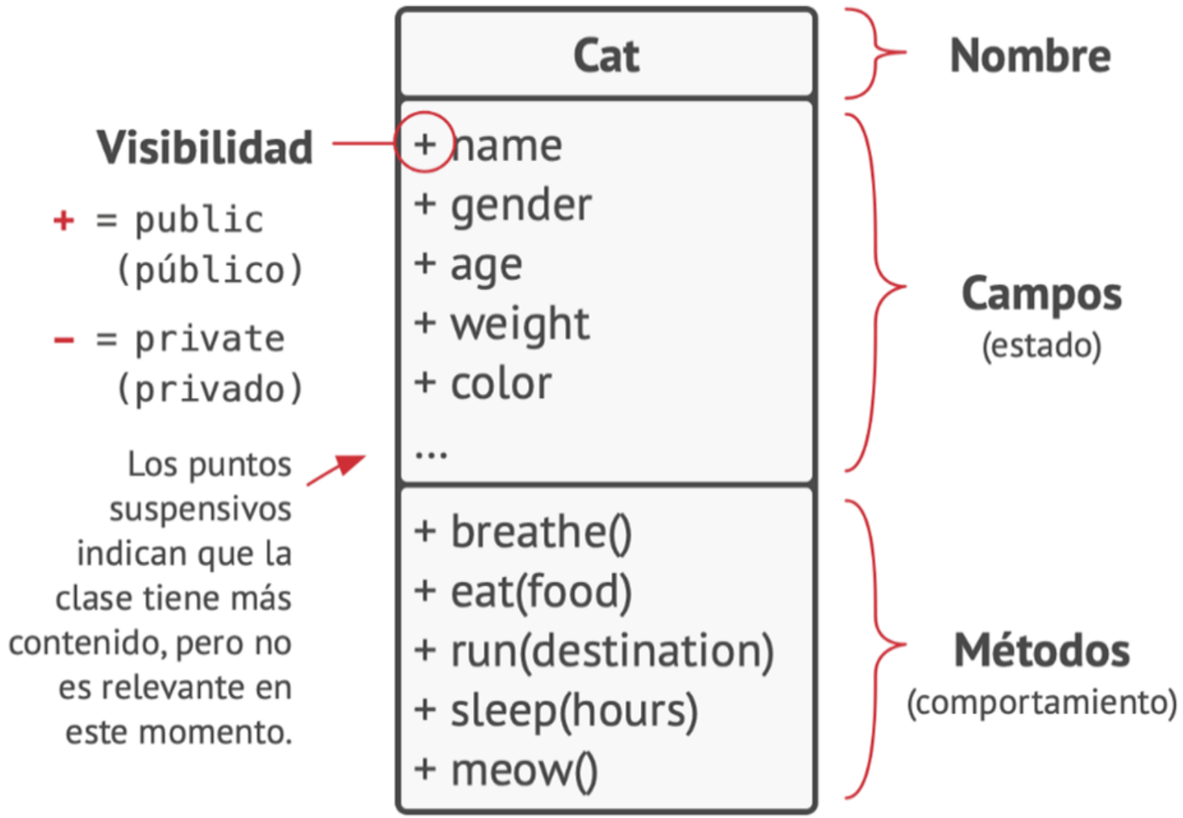


A foundational synthesis for advanced software design.

Atomic Units: The Blueprint and the Instance



Classes define the architectural constraints. Objects are the runtime realization of those constraints in memory.



Óscar: Cat

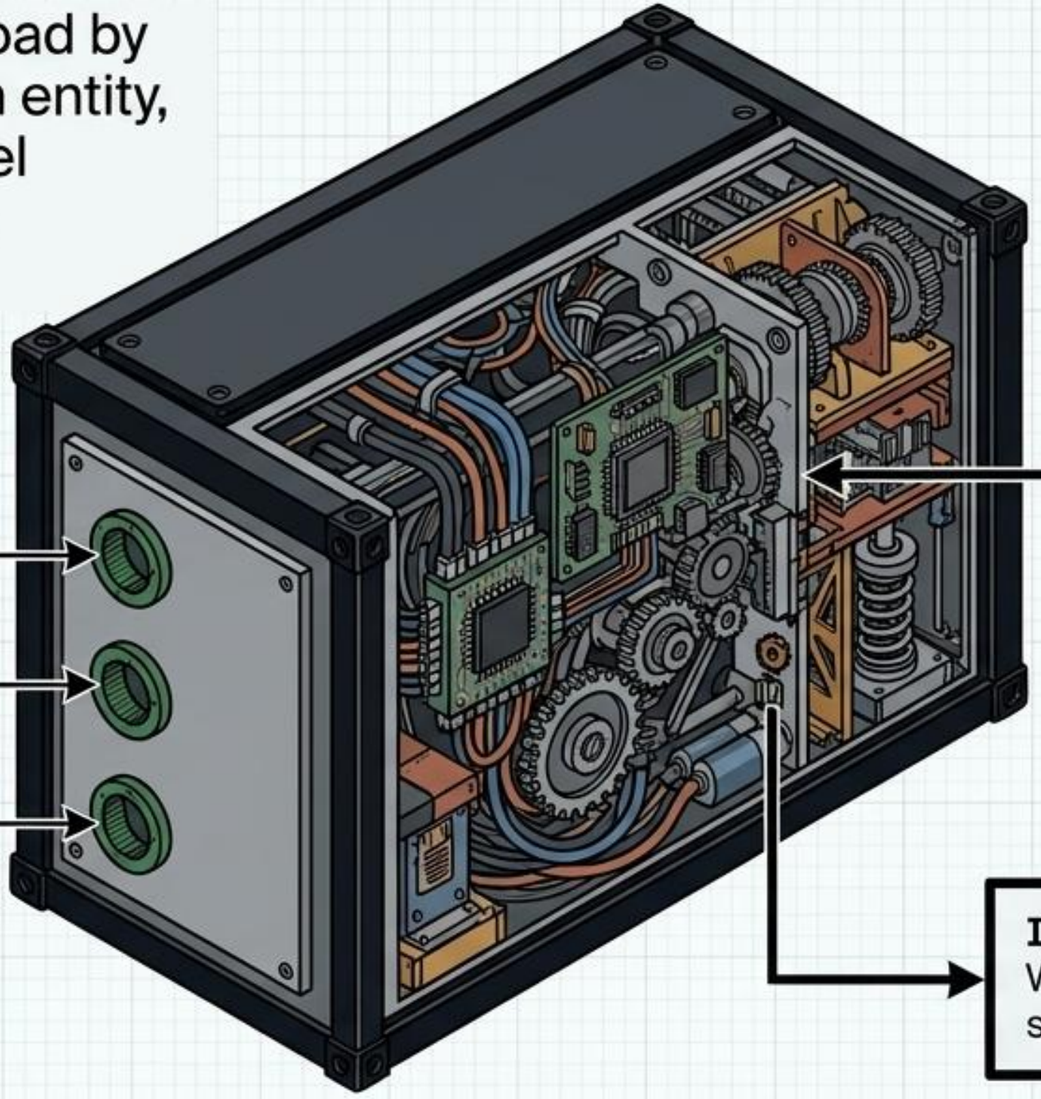
```
name    = "Óscar"
sex     = "macho"
age     =  3
weight  =  7
color   =  marrón
texture =  rayada
```

Luna: Cat

```
name    = "Luna"
sex     = "hembra"
age     =  2
weight  =  5
color   =  gris
texture =  lisa
```

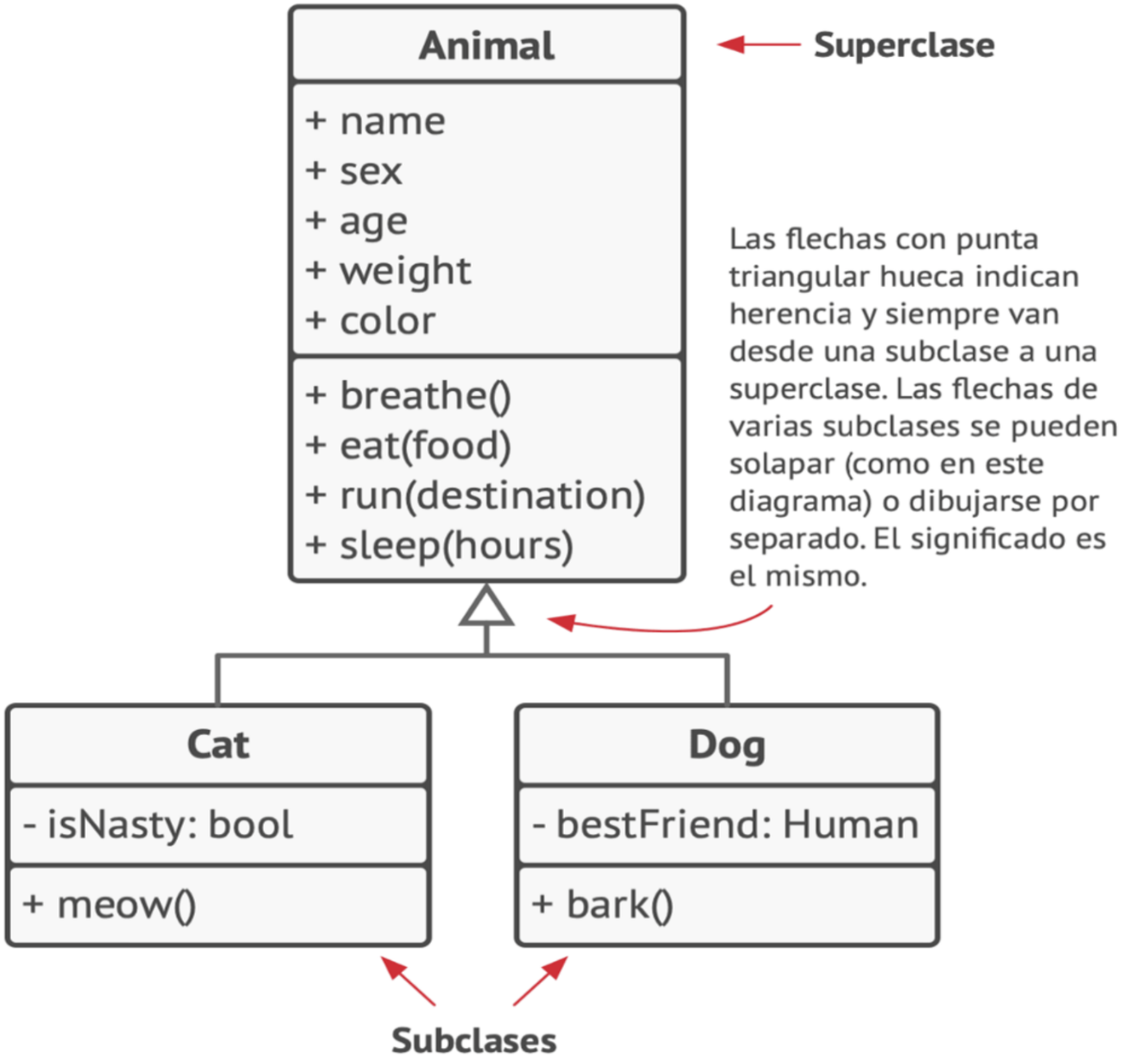
Pillar I: Abstraction as Complexity Management

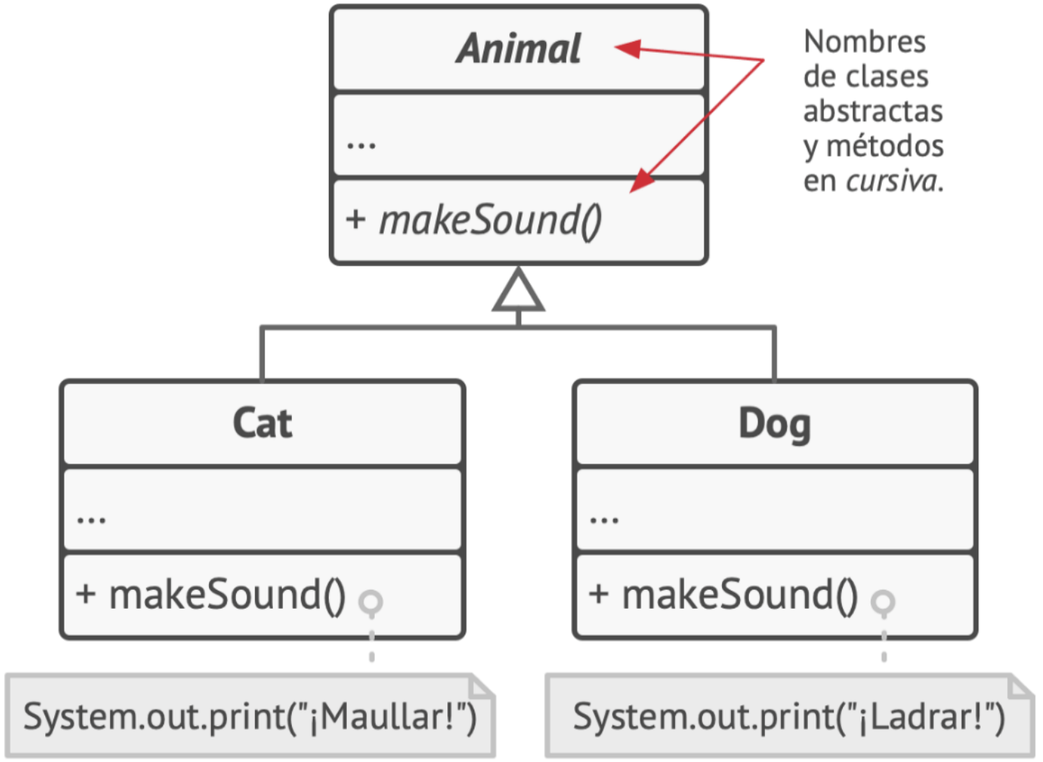
Abstraction reduces cognitive load by hiding the structural reality of an entity, surfacing only a high-level model relevant to the system's current context.



The Abstraction:
A simplified, stable contract exposing only essential operations.

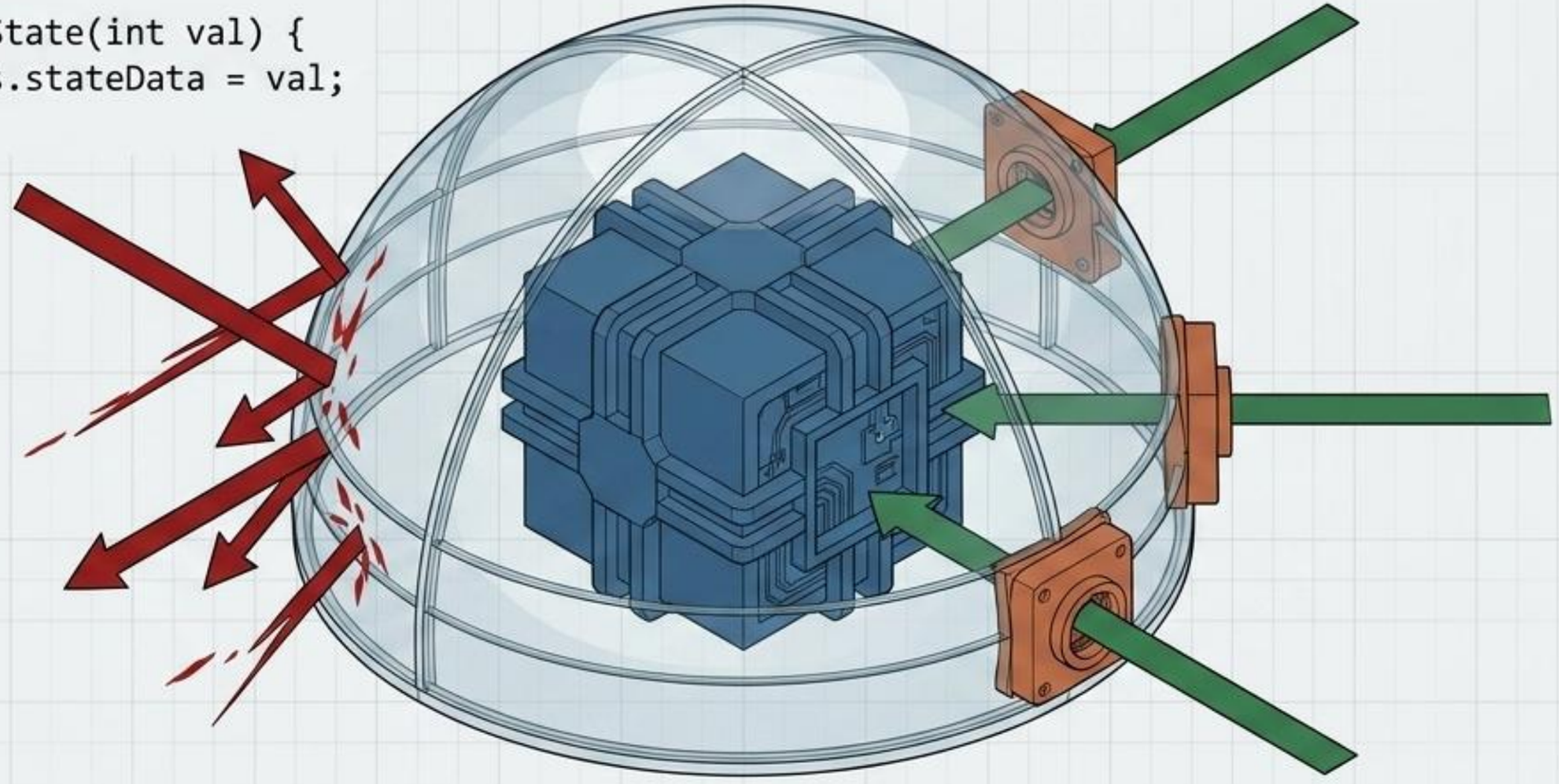
Implementation Details:
Volatile, complex, and subject to change.


Animal
+ name
+ sex
+ age
+ weight
+ color
+ breathe()
+ eat(food)
+ run(destination)
+ sleep(hours)
Superclase
Las flechas con punta triangular hueca indican herencia y siempre van desde una subclase a una superclase. Las flechas de varias subclases se pueden solapar (como en este diagrama) o dibujarse por separado. El significado es el mismo.
Cat
- isNasty: bool
+ meow()
Dog
- bestFriend: Human
+ bark()
Subclases

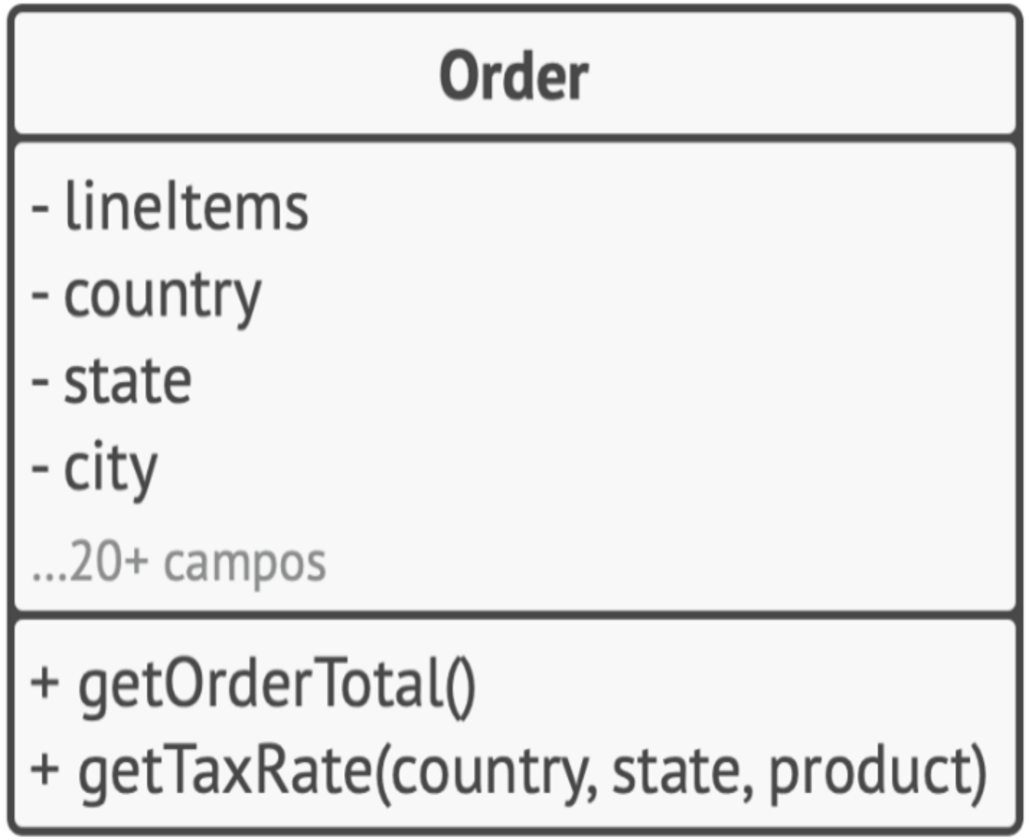

Animal
...
+ makeSound()
Nombres de clases abstractas y métodos en cursiva.
Cat
...
+ makeSound()
Dog
...
+ makeSound()
System.out.print("¡Maullar!")
System.out.print("¡Ladrar!")

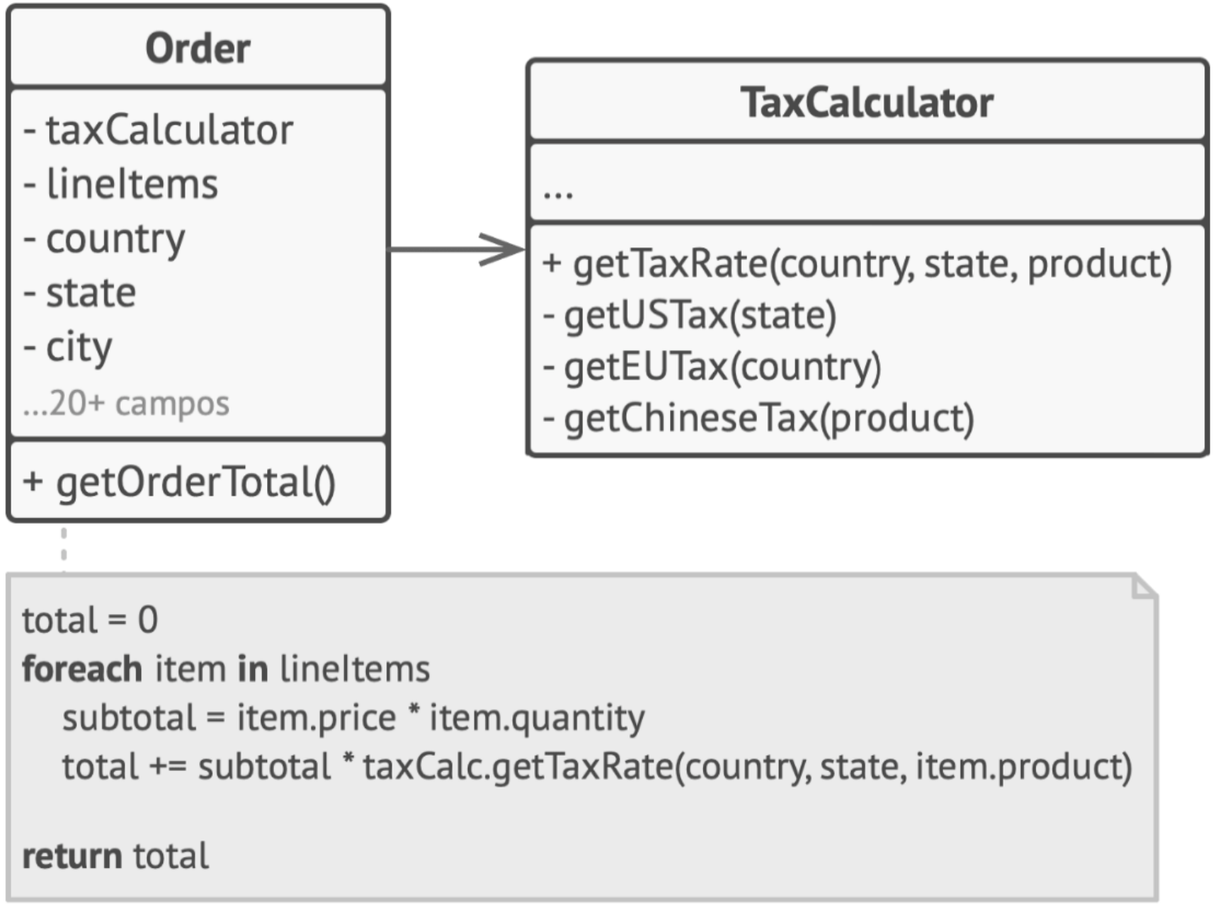
Pillar II: Encapsulation and State Shielding

```
private int stateData;

public void updateState(int val) {
  if (val > 0) this.stateData = val;
}
```



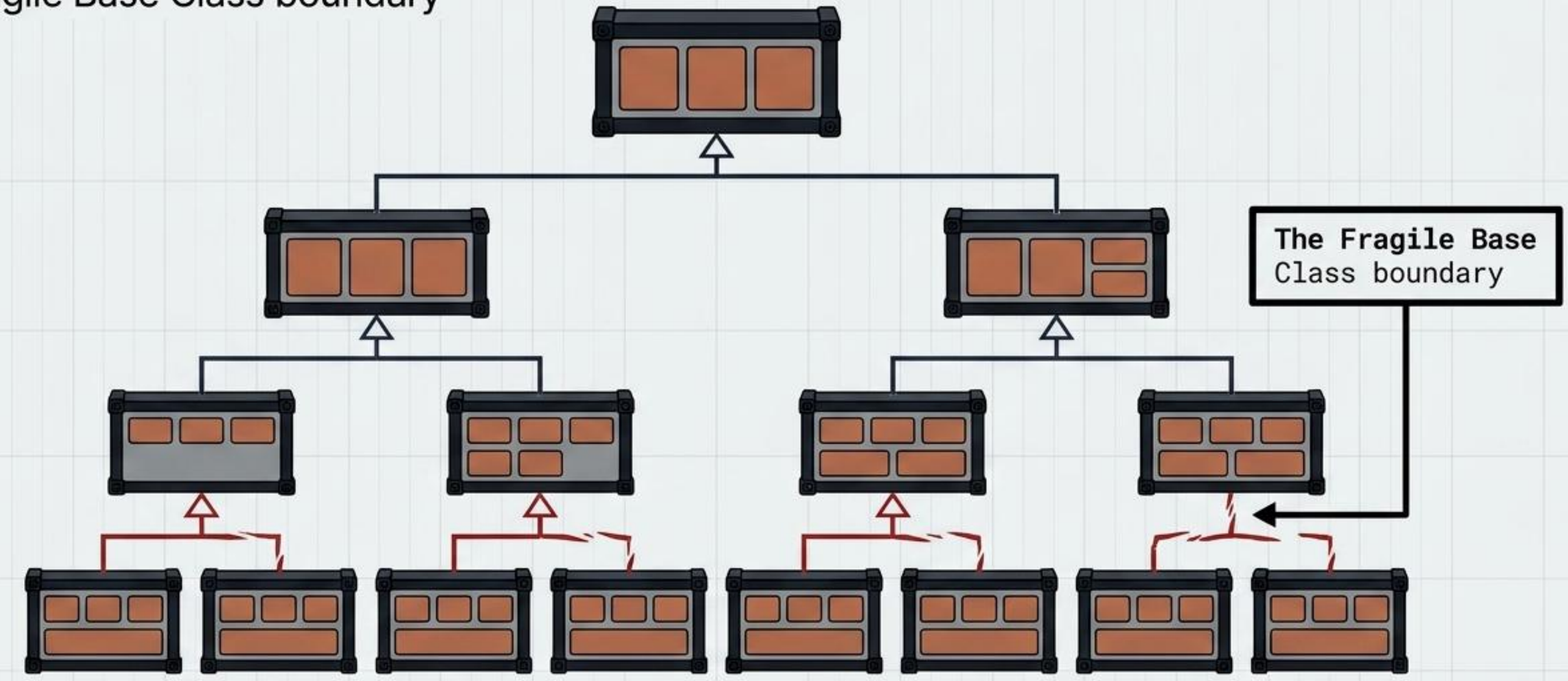
Encapsulation is not just data hiding; it is the active defense of an object's invariants. **State** must only be mutated by authorized **Behavior**.


Order
- lineItems
- country
- state
- city
...20+ campos
+ getOrderTotal()
+ getTaxRate(country, state, product)


Order
- taxCalculator
- lineItems
- country
- state
- city
...20+ campos
+ getOrderTotal()
TaxCalculator
...
+ getTaxRate(country, state, product)
- getUSTax(state)
- getEUTax(country)
- getChineseTax(product)
total = 0
foreach item in lineItems
subtotal = item.price * item.quantity
total += subtotal * taxCalc.getTaxRate(country, state, item.product)
return total

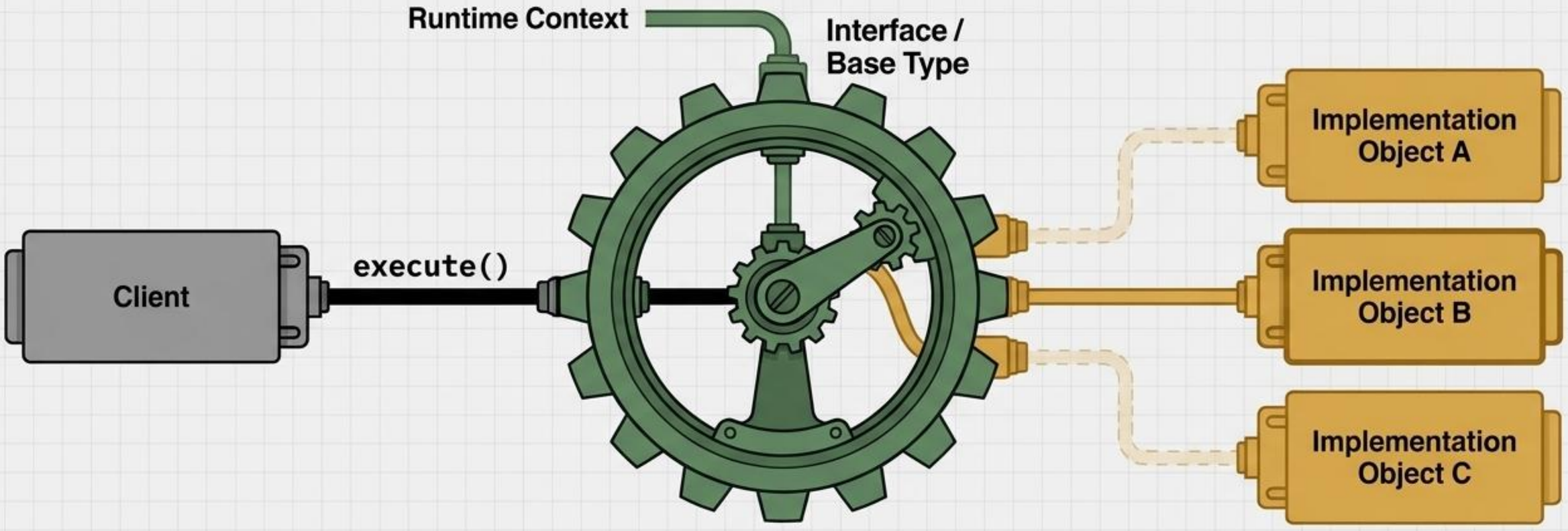
Pillar III: Inheritance and Structural Taxonomy

The Fragile Base Class boundary



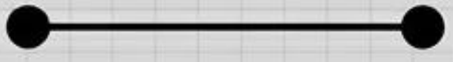
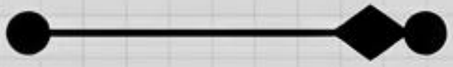
Inheritance establishes an ‘IS-A’ relationship, driving massive code reuse through structural hierarchy. However, deep nesting tightly couples derived classes to their parents, making the architecture brittle to upstream changes.

Pillar IV: Polymorphism and Dynamic Dispatch



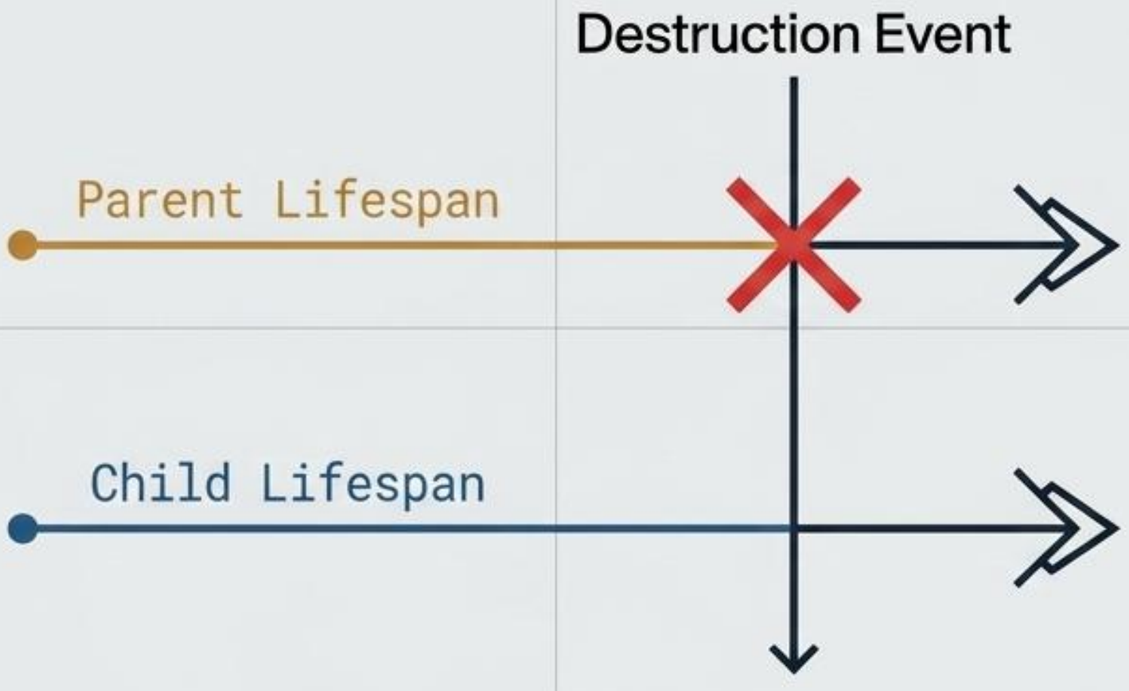
Polymorphism allows a single client interface to dictate multiple, interchangeable underlying behaviors. The sender knows what to command, but is entirely decoupled from how it is executed.

The Object Relationship Spectrum

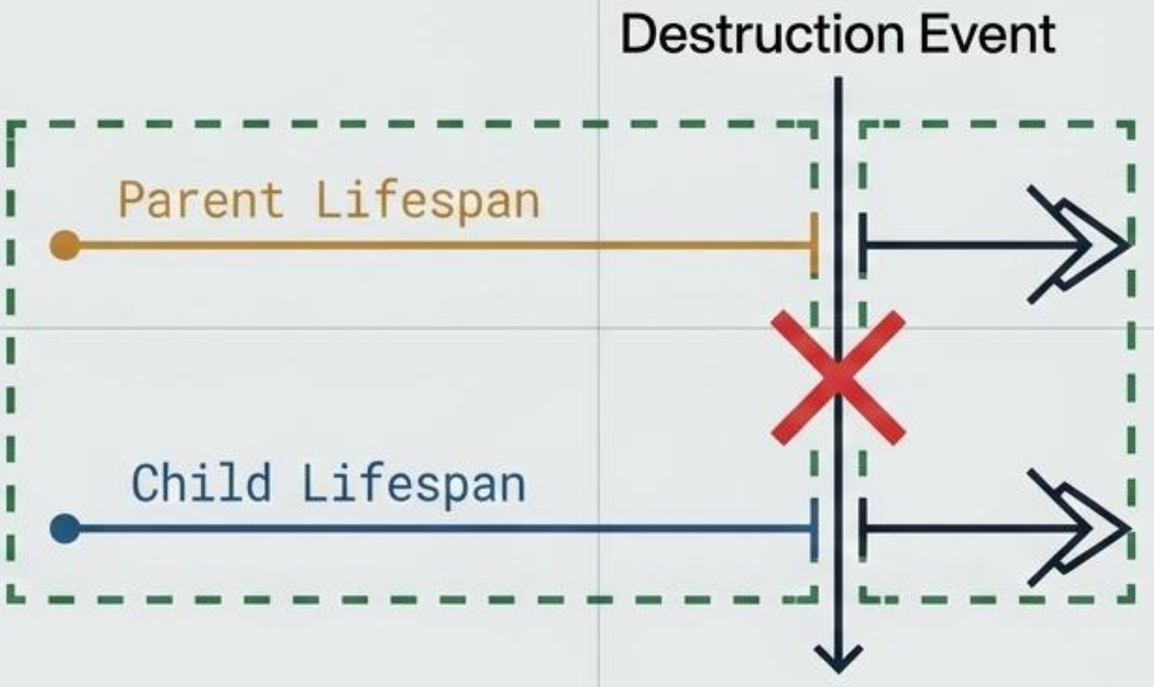
RELATIONSHIP	ASSOCIATION	AGGREGATION	COMPOSITION
Coupling	Weak	Medium	Strong
Lifecycle	Independent	Independent (Part can survive the Whole)	Co-dependent (Whole destroys the Part)
UML Symbol			
Analogy	Teachers and Students in a room	A University Department and its Professors	A House and its Rooms

Decoupling Lifecycles: Aggregation vs. Composition

Aggregation



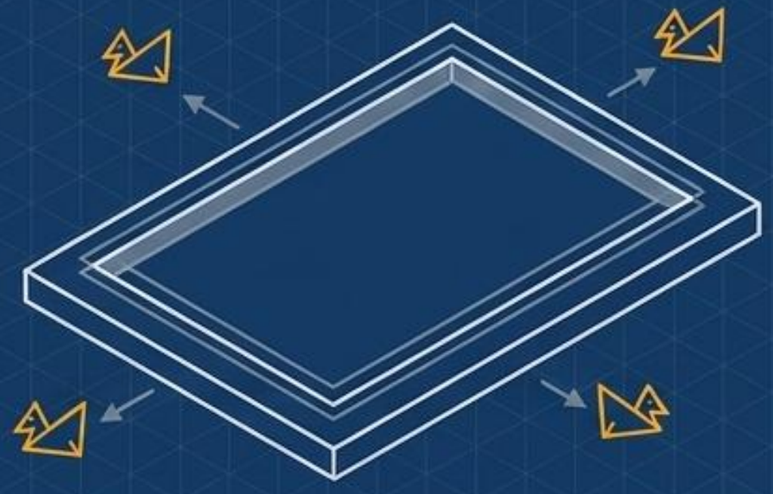
Composition



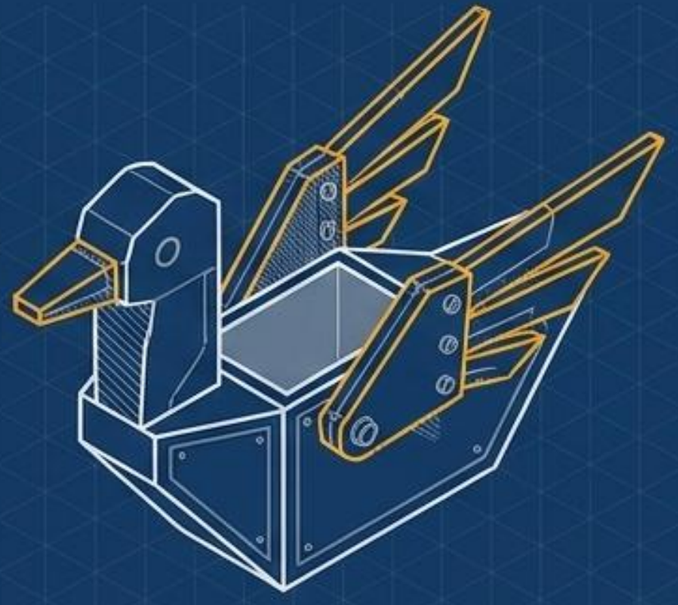
The definitive test of Composition is lifecycle sovereignty. If the parent's garbage collection strictly necessitates the child's garbage collection, it is Composition.

Deep Dive: Lifecycles and Whole-Part Hierarchies

Aggregation (Hollow Diamond): The Pond has Ducks. If the Pond dries up (is destroyed), the Ducks simply fly away. They have distinct, independent lifecycles.



Composition (Solid Diamond): The Duck owns Wings. If the Duck is destroyed, the Wings are destroyed with it. They share a strict, unified lifecycle.



The Spectrum of Coupling

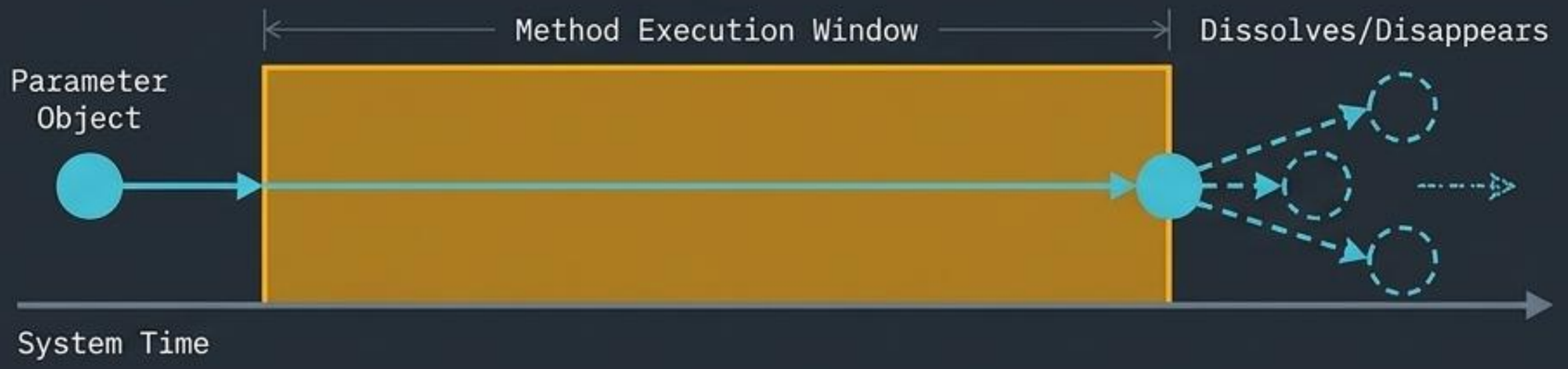
Not all object interactions carry the same structural weight. Evaluating software architecture requires understanding the permanence and strength of relationships. We move from fleeting, method-level interactions to strict, class-level contracts.

Dependency: The ‘Uses-A’ Relationship

Definition: A relationship where one `class` relies on another class for a specific, limited operation. Changes to the `supplier class` may affect the `client class`.

Key Insight: Dependency is the **weakest** form of coupling in object-oriented design. The `client` object has no structural reliance on the `supplier` **object** outside of a brief moment of interaction, ensuring independent lifecycles.

The Anatomy of a Temporary Bond



Dependencies exist only in fleeting scopes:

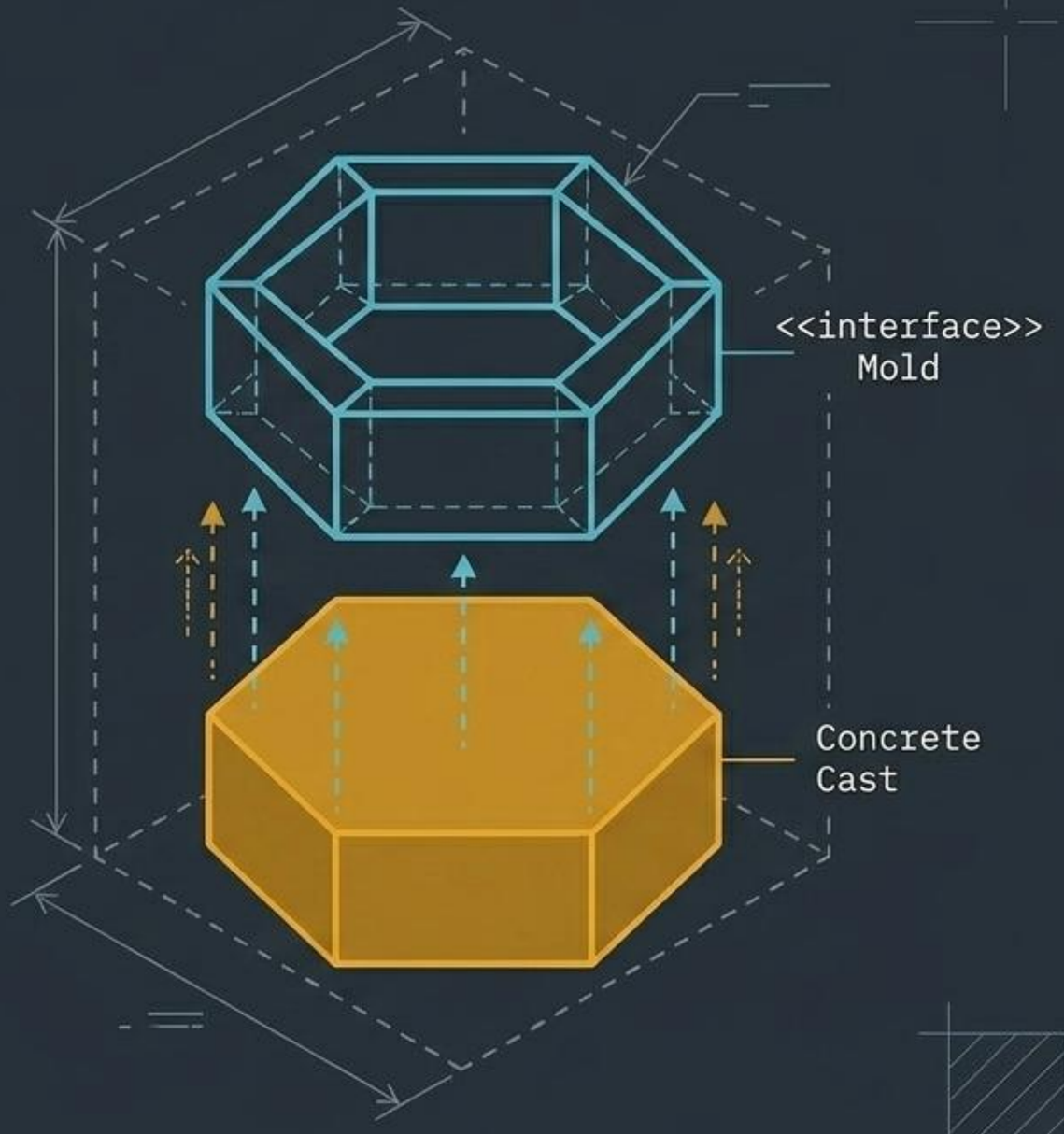
1. Method parameters
2. Local variables
3. Return types

```
1 public void calculateTotal(DiscountStrategy strategy) {
2     // Dependency exists ONLY during this method execution
3     double total = strategy.apply(this.basePrice);
4 }
```

Realization: The Architectural Contract

Definition: The structural relationship between an abstraction (`interface`) and a `concrete class` that implements it.

Core Concept: Realization shifts the relationship from an action (uses) to an identity (`fulfills`). The concrete class signs a contract to provide specific behavior, bound at `compile-time`.



Abstraction vs. Concrete Implementation

The Promise (Interface)

Defines Contract, No Behavior

```java
interface PaymentProcessor {
    void process(double amount);
    boolean verifyAccount();
}
```

The Fulfillment (Concrete Class)

Provides Specific Behavior, Fulfills Contract

```java
class StripeProcessor implements PaymentProcessor {
    public void process(double amount) {
        // Concrete implementation logic
    }

    public boolean verifyAccount() {
        // Concrete verification logic
    }
}
```

Visual Notation Guide: Reading the Blueprint

Dependency

Indicates a client uses a supplier. The open arrowhead points to the independent element that is being used.

Realization

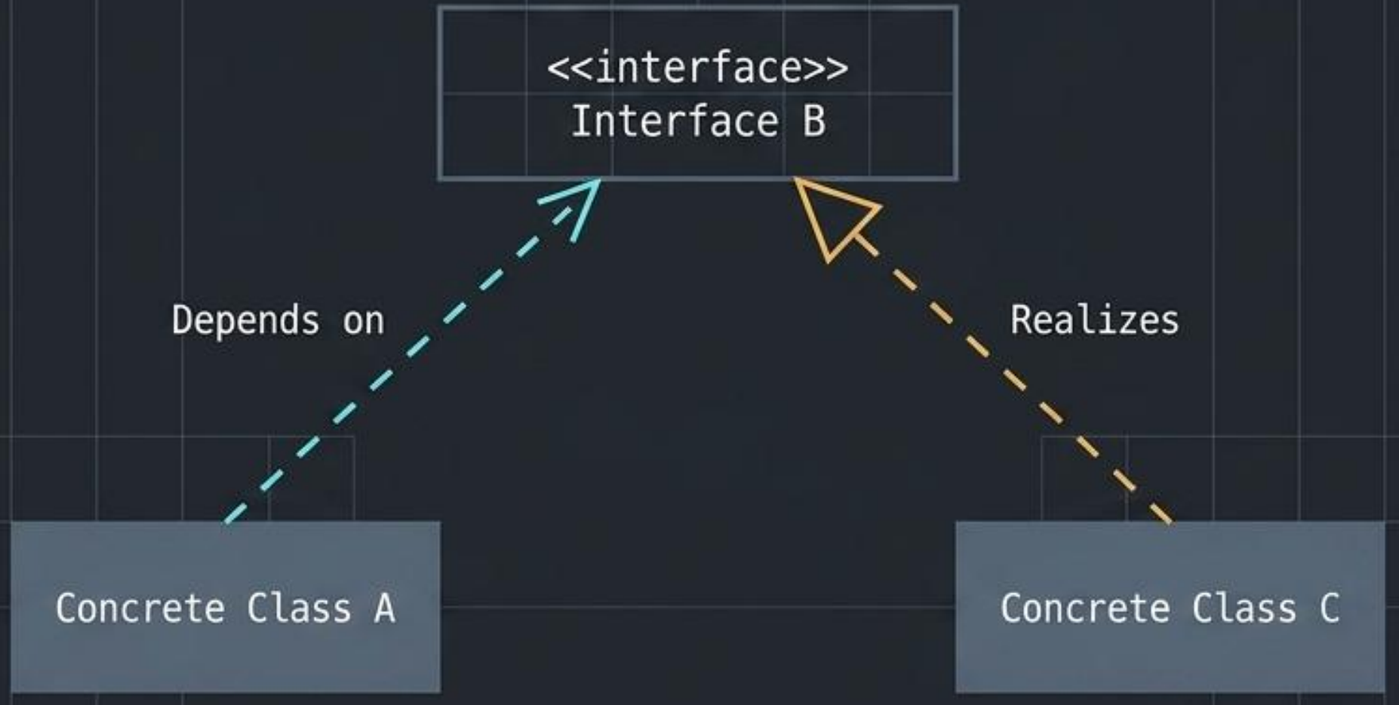
Indicates a class implements an interface. The hollow triangle points directly to the abstraction being realized.

Diagnostic Matrix: Analyzing the Blueprint

Dimension	Dependency	Realization
Nature / Concept	"Uses-a" action	"Fulfills-a" identity
Lifecycle Permanence	Temporary (Method Execution)	Permanent (Compile-time)
Coupling Strength	Weakest form of coupling	Strict, binding contract
UML Symbol	`Dashed Line + Open Arrow`	`Dashed Line + Hollow Triangle`
Code Trigger	Parameter, Local Variable	`implements` keyword

Synthesis: Program to an interface, not an implementation

The ultimate design pattern principle relies on combining both relationships. By establishing a Dependency on an Abstraction, rather than a concrete class, the system remains entirely agnostic to the underlying Realization. This is the foundation of the Strategy, Factory, and Observer patterns.



The Decoupled Future

PANEL 1

Keep it Weak

Default to Dependency. It is the lightest, safest way for objects to collaborate without entangling their lifecycles.

PANEL 2

Bind to Abstractions

Use Realization to define strict structural contracts and promises.

PANEL 3

Master the Blueprint

Recognizing the visual and conceptual difference between an open arrow and a hollow triangle is the first step toward scalable, refactorable system design.

The Coupling Spectrum

Favor object composition over class inheritance.

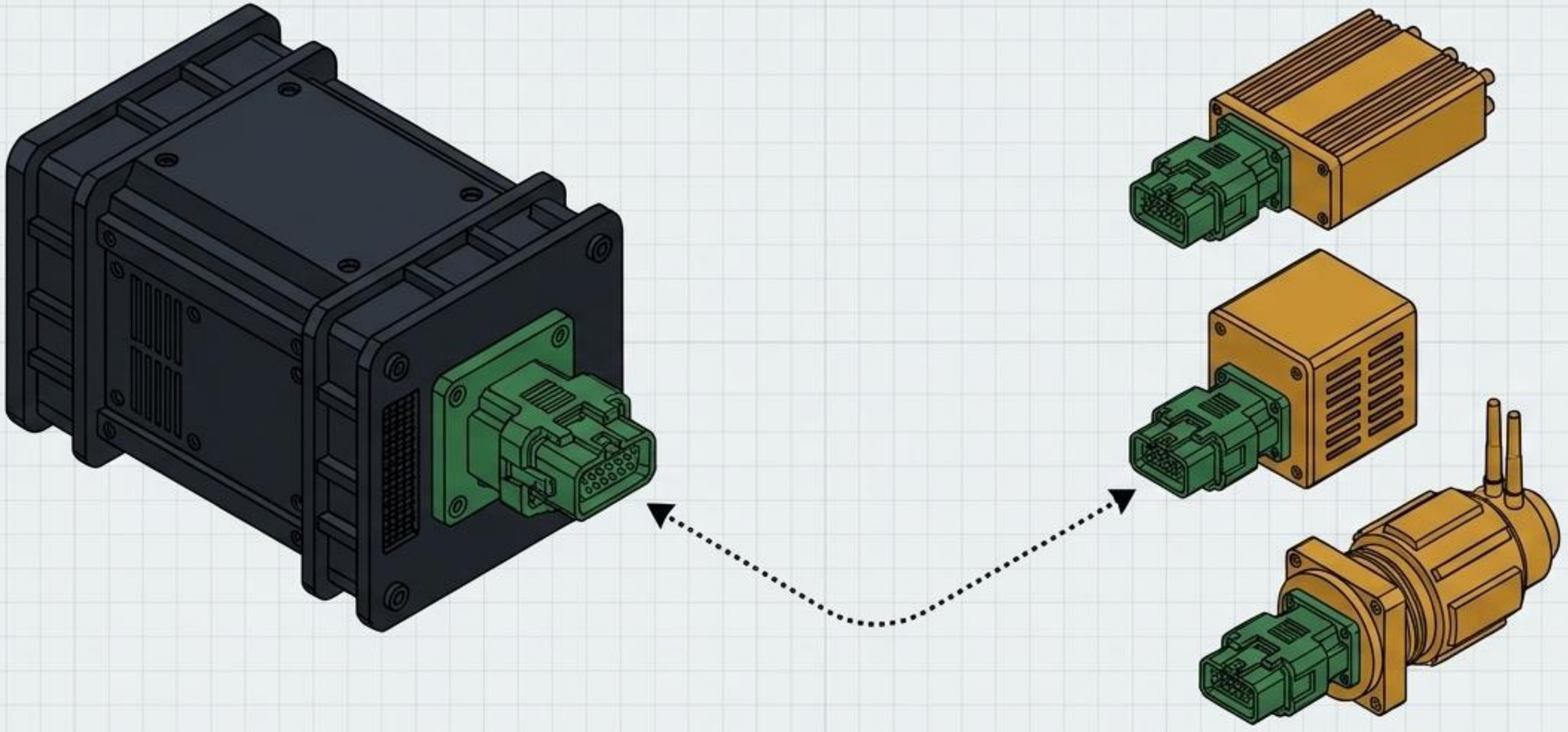
Visual Notation	Semantic Meaning	Coupling Strength
	Dependency (Uses-a)	 Very Loose
	Aggregation (Has-a)	 Moderate
	Composition (Owns-a)	 Tight
	Inheritance (Is-a)	 Very Tight

"Favor object composition over class inheritance."

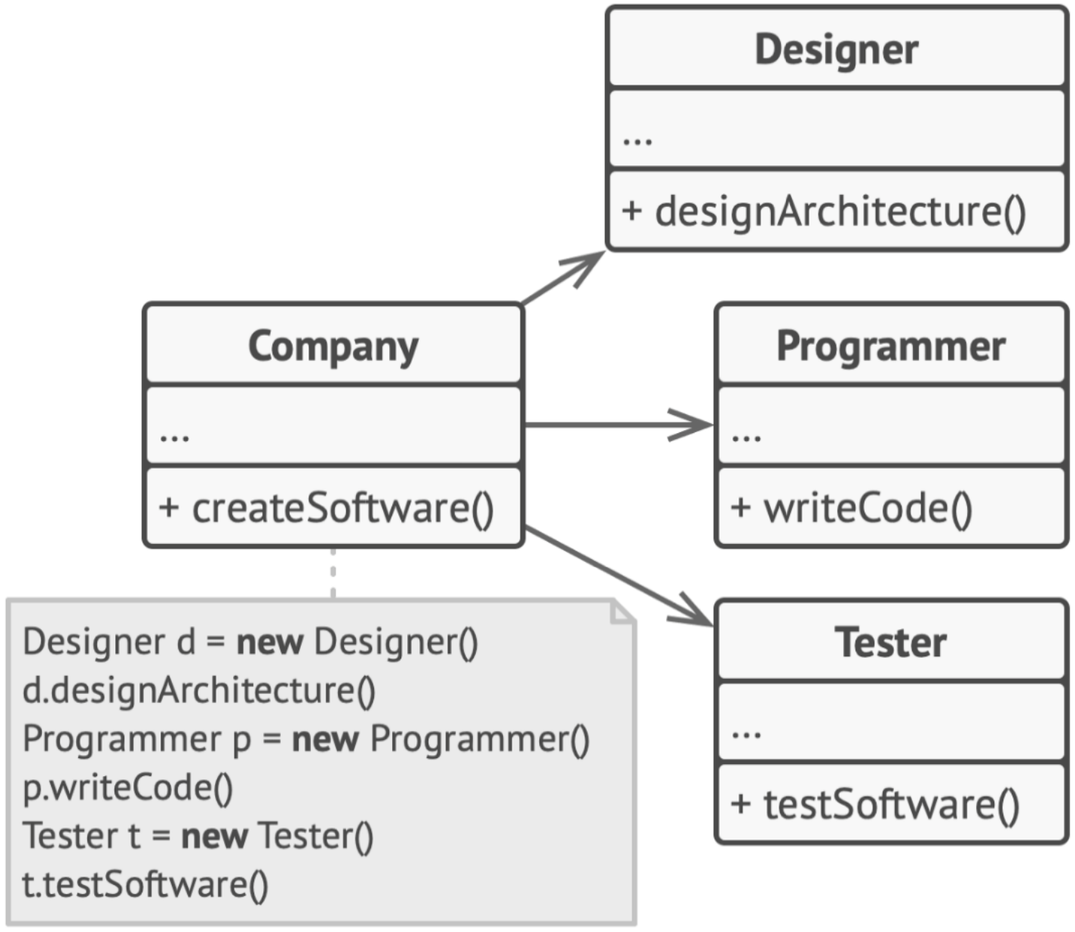
GoF Principle I: Program to an Interface, Not an Implementation

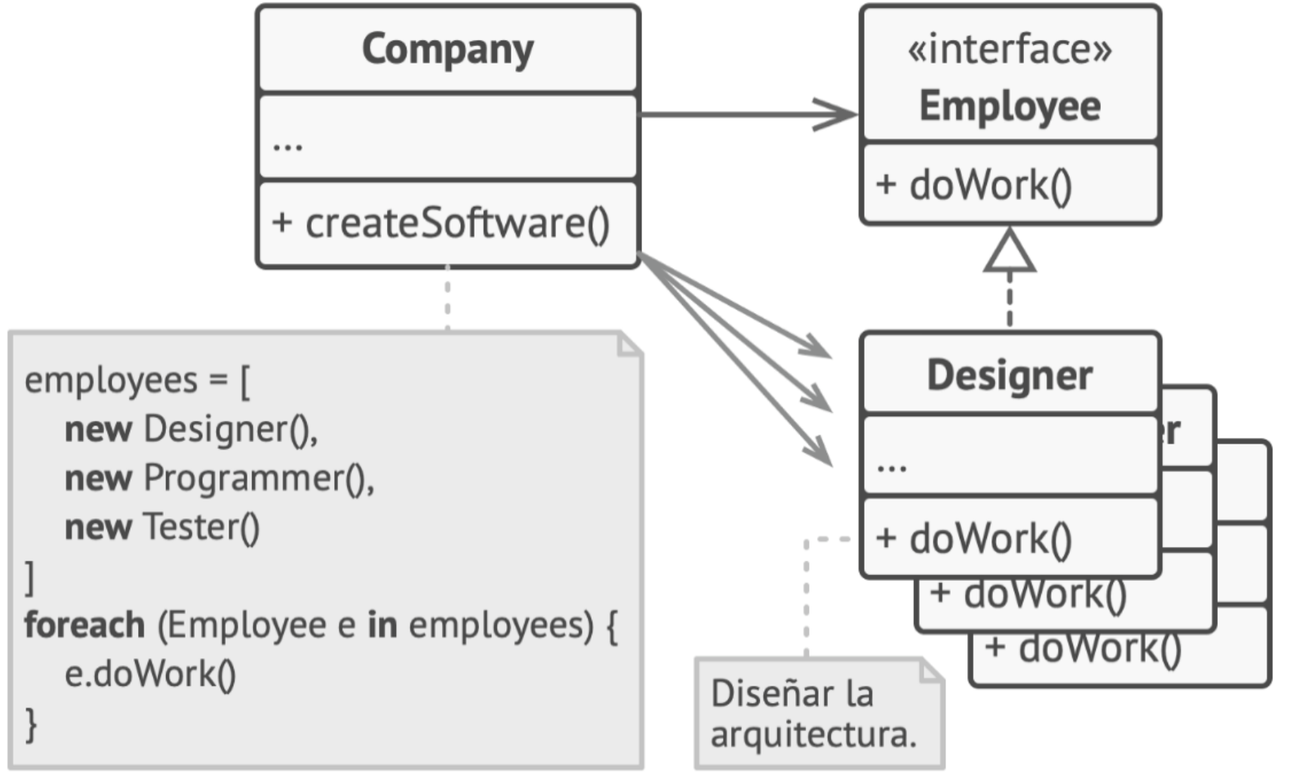
The Interface Contract

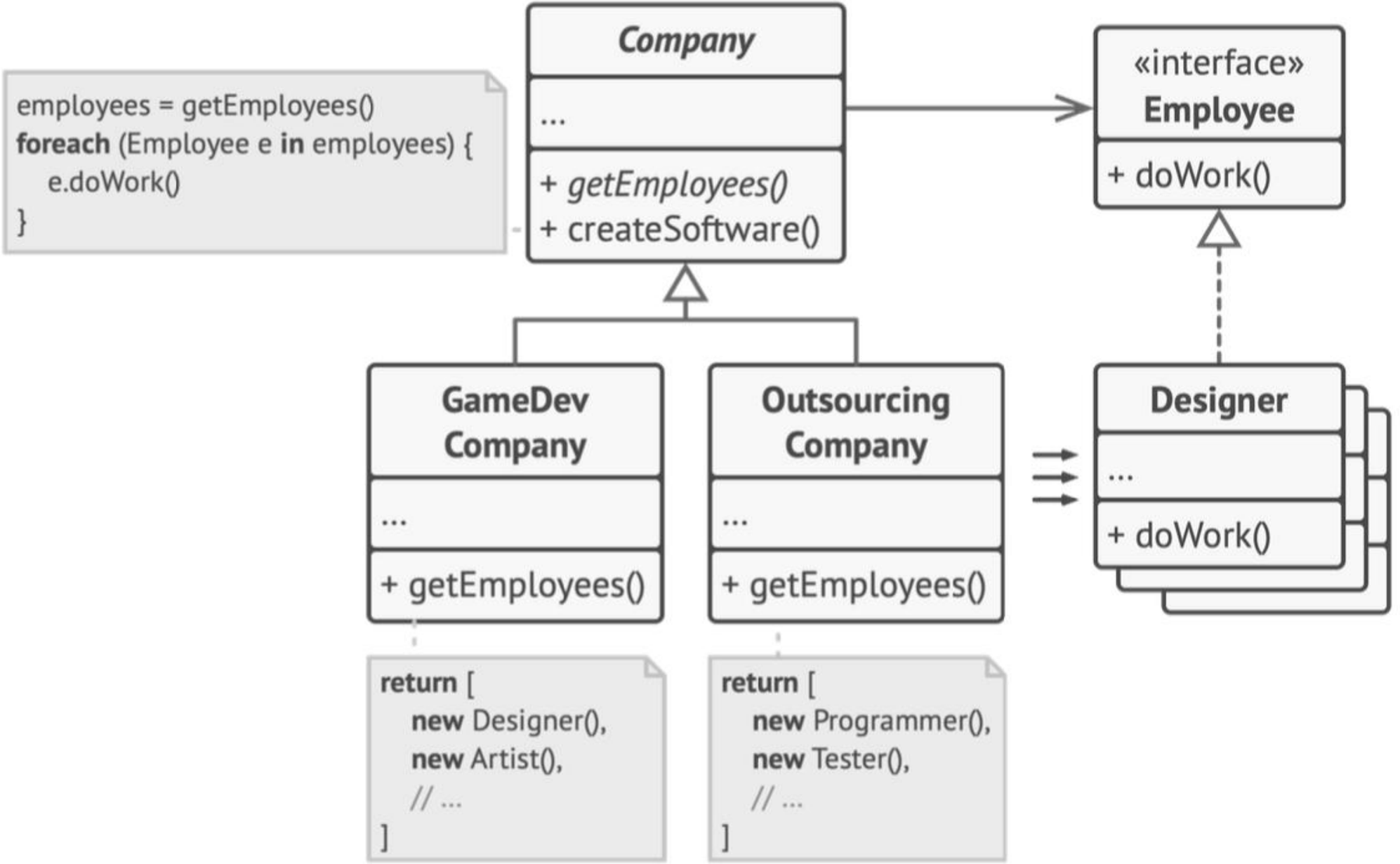
Implementation Modules



By binding dependencies to an abstract interface rather than a concrete class, systems decouple the ‘intent’ from the ‘mechanism’. This is the architectural foundation of open-closed extensibility


Designer
...
+ designArchitecture()
Company
...
+ createSoftware()
Programmer
...
+ writeCode()
Tester
...
+ testSoftware()
Designer d = new Designer()
d.designArchitecture()
Programmer p = new Programmer()
p.writeCode()
Tester t = new Tester()
t.testSoftware()

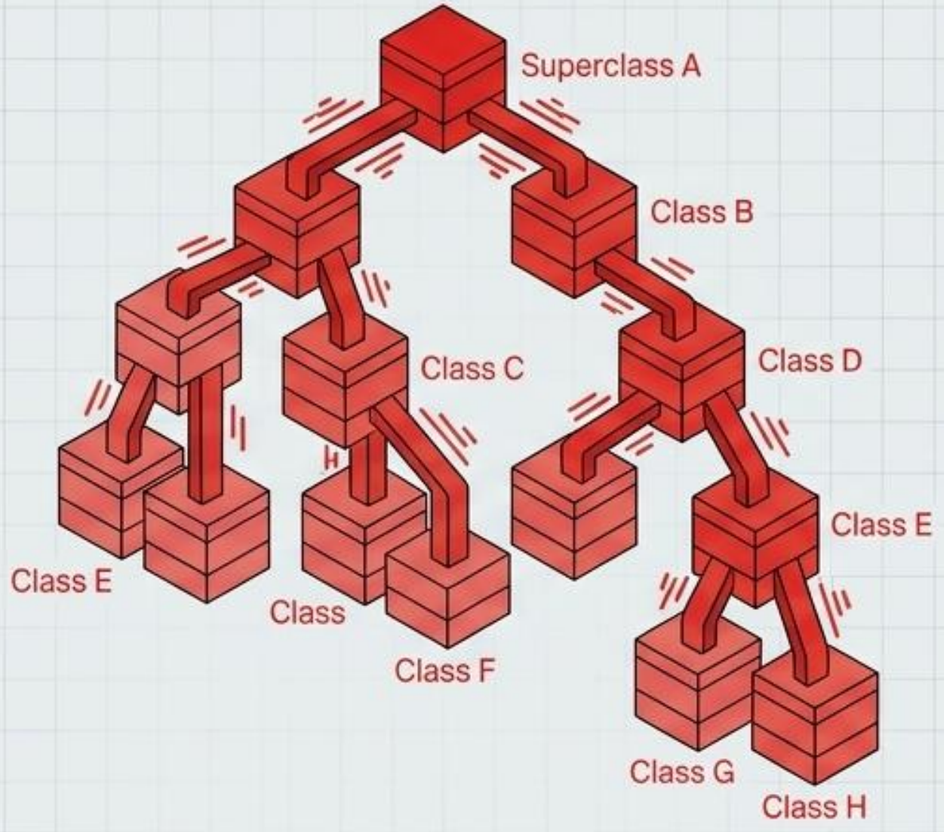

Company
...
+ createSoftware()
«interface»
Employee
+ doWork()
Designer
...
+ doWork()
+ doWork()
+ doWork()
employees = [
new Designer(),
new Programmer(),
new Tester()
]
foreach (Employee e in employees) {
e.doWork()
}
Diseñar la arquitectura.


employees = getEmployees()
foreach (Employee e in employees) {
e.doWork()
}
Company
...
+ getEmployees()
+ createSoftware()
«interface»
Employee
+ doWork()
GameDev
Company
...
+ getEmployees()
Outsourcing
Company
...
+ getEmployees()
Designer
...
+ doWork()
return [
new Designer(),
new Artist(),
// ...
]
return [
new Programmer(),
new Tester(),
// ...
]

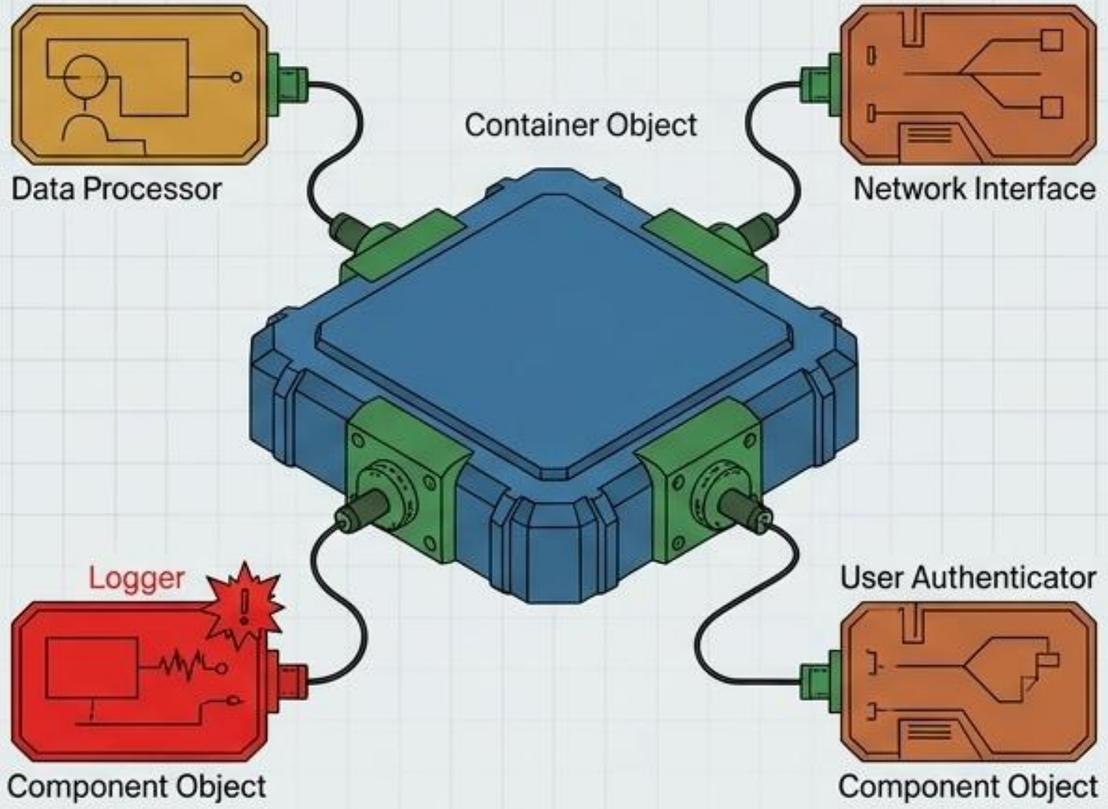
GoF Principle II: Favor Composition Over Inheritance

Refactoring View

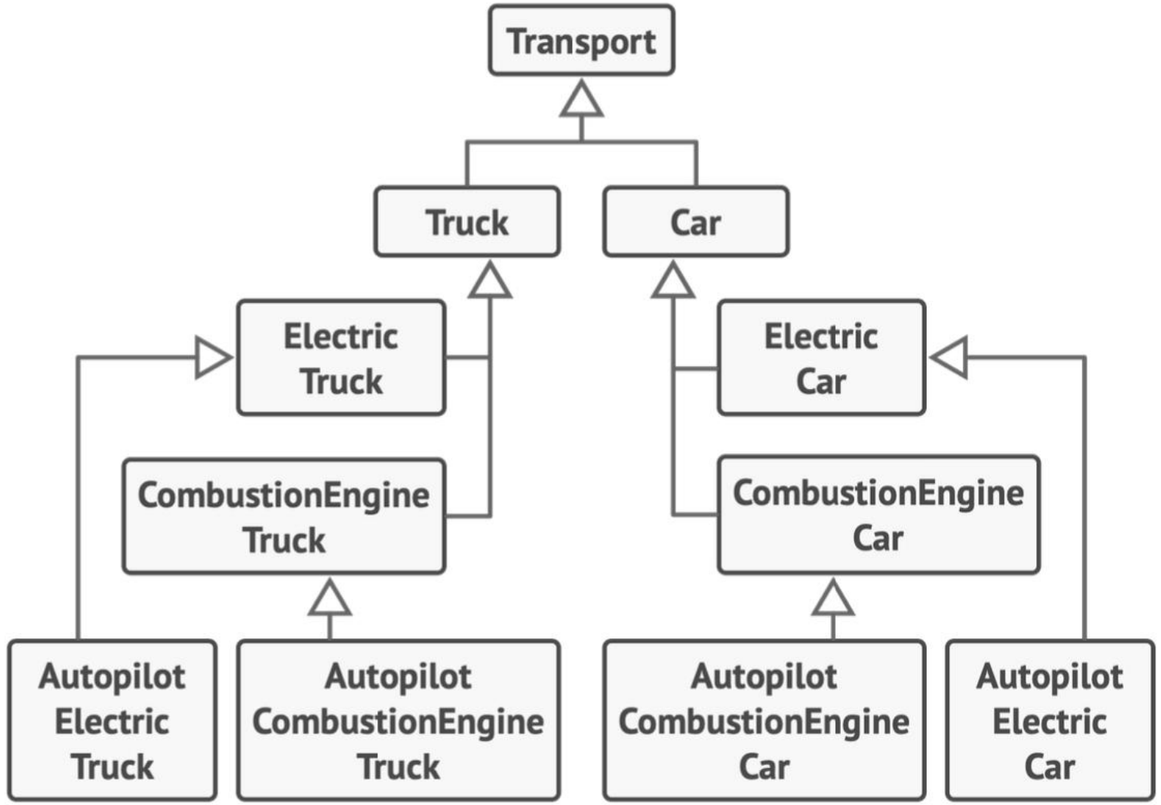
Inheritance: The Fragile Taxonomy

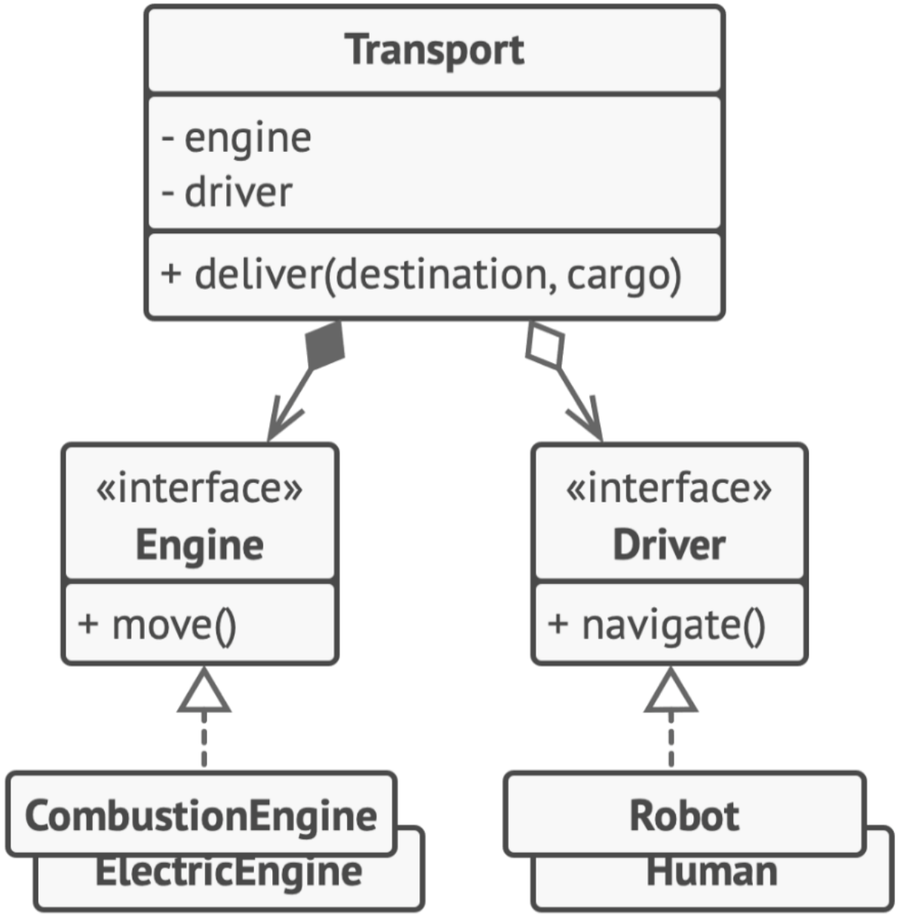


Composition: The Modular Assembly



Inheritance achieves reuse via rigid, compile-time ‘White-box’ taxonomies.
Composition achieves reuse via flexible, run-time ‘Black-box’ assembly.


Transport
Truck
Car
Electric Truck
Electric Car
CombustionEngine Truck
CombustionEngine Car
Autopilot Electric Truck
Autopilot CombustionEngine Truck
Autopilot CombustionEngine Car
Autopilot Electric Car

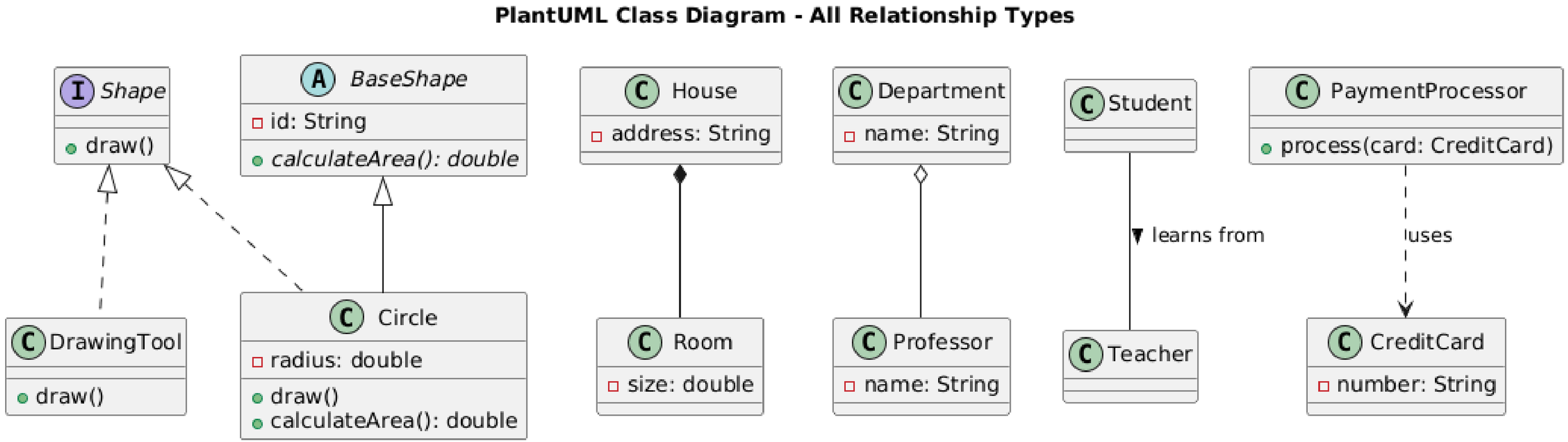

Transport
- engine
- driver
+ deliver(destination, cargo)
«interface» Engine
+ move()
«interface» Driver
+ navigate()
CombustionEngine
ElectricEngine
Robot
Human

The Architectural Diagnostic Scorecard

	INHERITANCE	COMPOSITION
Flexibility	Static / Compile-time	Dynamic / Run-time
Encapsulation	Breaks encapsulation; subclasses expose parent internals	Preserves strict encapsulation via interfaces
Unit Testing	Difficult; requires mocking entire deep hierarchies	Trivial; components can be mocked individually
Refactoring Friction	High; changes cascade downward	Low; isolated interface contracts

While inheritance is conceptually simpler, composition yields systems that are vastly more resilient to changing business requirements over time.

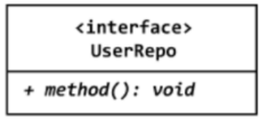
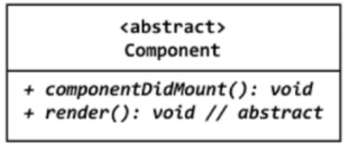
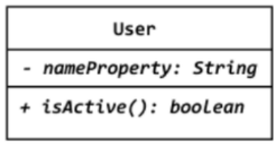
https://plantuml.com/class-diagram



```
@startuml
title PlantUML Class Diagram - All Relationship Types

' 1. Interface and Abstract Class
interface Shape {
    +draw()
}

abstract class BaseShape {
    -id: String
    {abstract} +calculateArea(): double
}

' 2. Inheritance (Generalization)
' BaseShape inherits from Shape (Realization)
' Circle inherits from BaseShape
class Circle extends BaseShape implements Shape {
    -radius: double
    +draw()
    +calculateArea(): double
}

' 3. Realization (Implementation)
' DrawingTool realizes the Shape interface logic
class DrawingTool implements Shape {
    +draw()
}

' 4. Composition (Strong "has-a" - Lifetime dependency)
' A House consists of Rooms. If House is deleted, Rooms are deleted.
class House {
    -address: String
}
class Room {
    -size: double
}
House *-- Room

' 5. Aggregation (Weak "has-a" - Independent lifetime)
' A Department has Professors, but Professors exist without the
Department.
class Department {
    -name: String
}
class Professor {
    -name: String
}
Department o-- Professor

' 6. Association (Simple relationship)
' A Student "is associated" with a Teacher.
class Student
class Teacher
Student -- Teacher : learns from >

' 7. Dependency (Uses-a - Temporary relationship)
' PaymentProcessor "uses" a CreditCard to process a transaction.
class PaymentProcessor {
    +process(card: CreditCard)
}
class CreditCard {
    -number: String
}
PaymentProcessor ..> CreditCard : uses

@enduml
```

UML Class Diagram Cheatsheet

khalilstemmler.com

Shape	Description
	**Package** A collection of classes and interfaces.
	**Interface** Interface name written underneath the <interface> annotation. Methods underneath.
	**Abstract class** Same as the interface shape. Abstract methods marked as abstract with comments or "abstract methodName(): returnType".
	**Class** Properties or attributes sit at the top, methods or operations at the bottom + indicates public, - indicates private, and # indicates protected

These should be drawn vertically

Inheritance

B inherits from A. Creates an "is-a" relationship. A is a generalization.

Implementation/realization

B is a concrete implementation/realization of A.

Association

A and B call each other.

One way association

A can call B's properties/methods, but not vice versa.

Aggregation

A has 1 or more instances of B. B can survive if A is disposed.

Ex: Professor (1) "has-many" classes (0..) to teach.*

Ex: Pond (0..1) "has-many" ducks (0..). Ducks can survive if the pond is destroyed.*

Composition

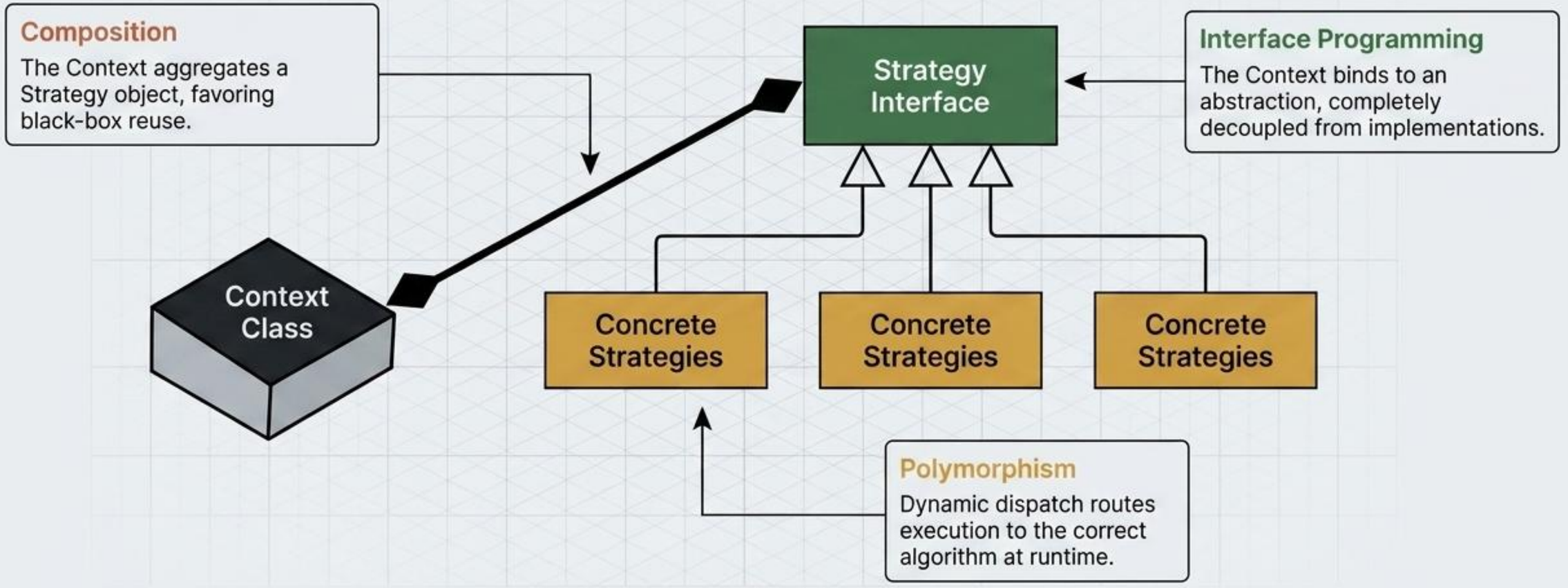
A has 1 or more instances of B. B cannot survive if A is disposed.

Ex: User (1) "has a" UserName (1). UserNames can't exist as separate parts in away from a User in our application.

Note

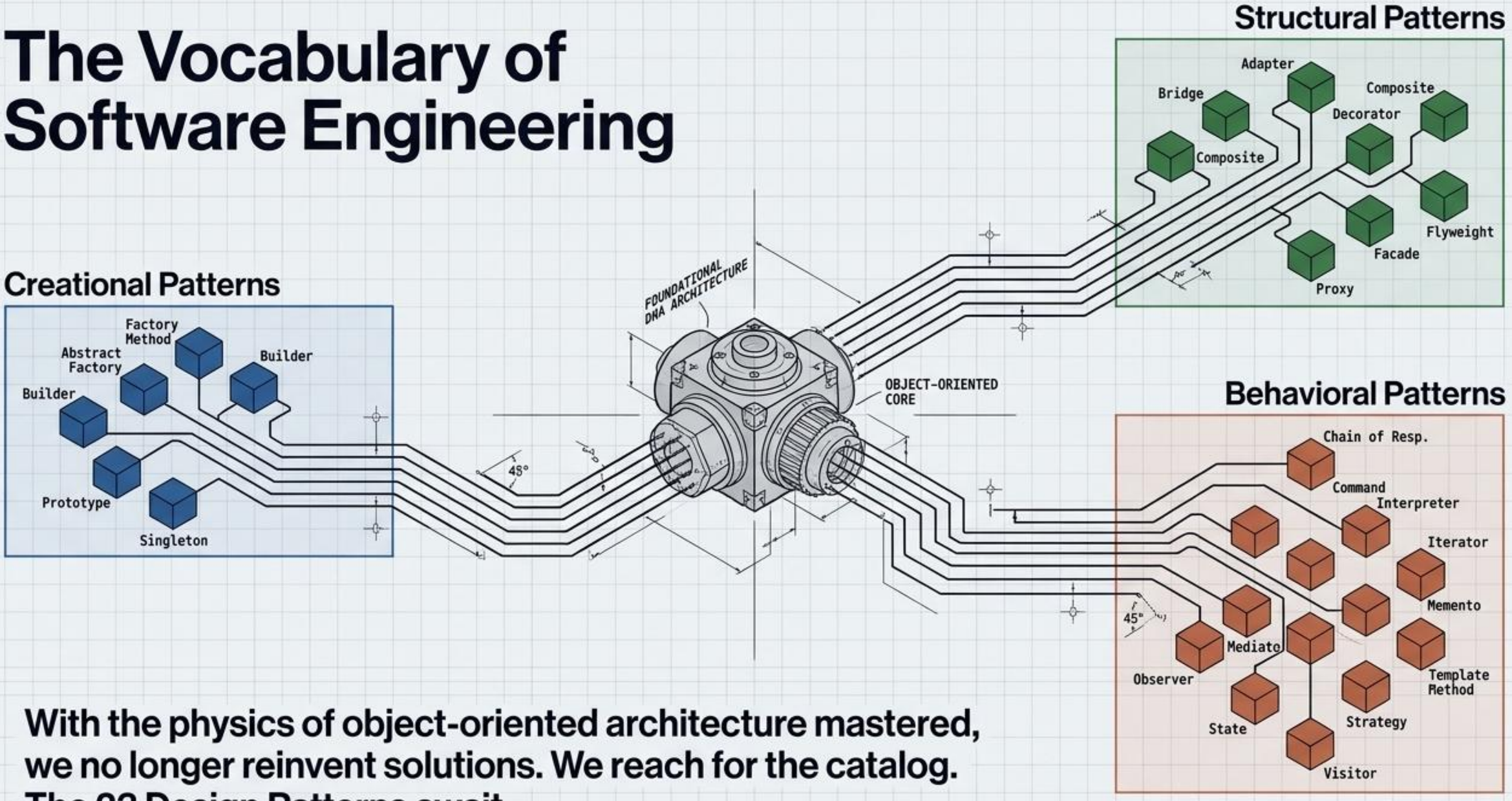
Descriptive text that can be attached to any item.

Synthesis: The DNA of a Design Pattern



Every GoF Design Pattern is fundamentally just a calculated arrangement of Polymorphism, Composition, and Interface Programming.

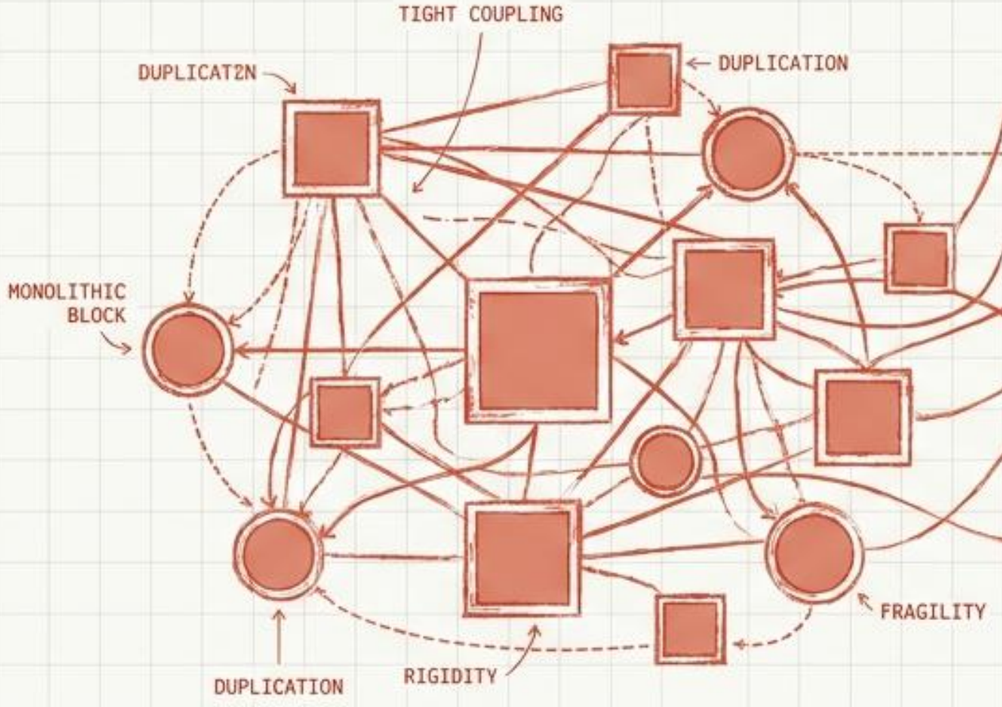
The Vocabulary of Software Engineering

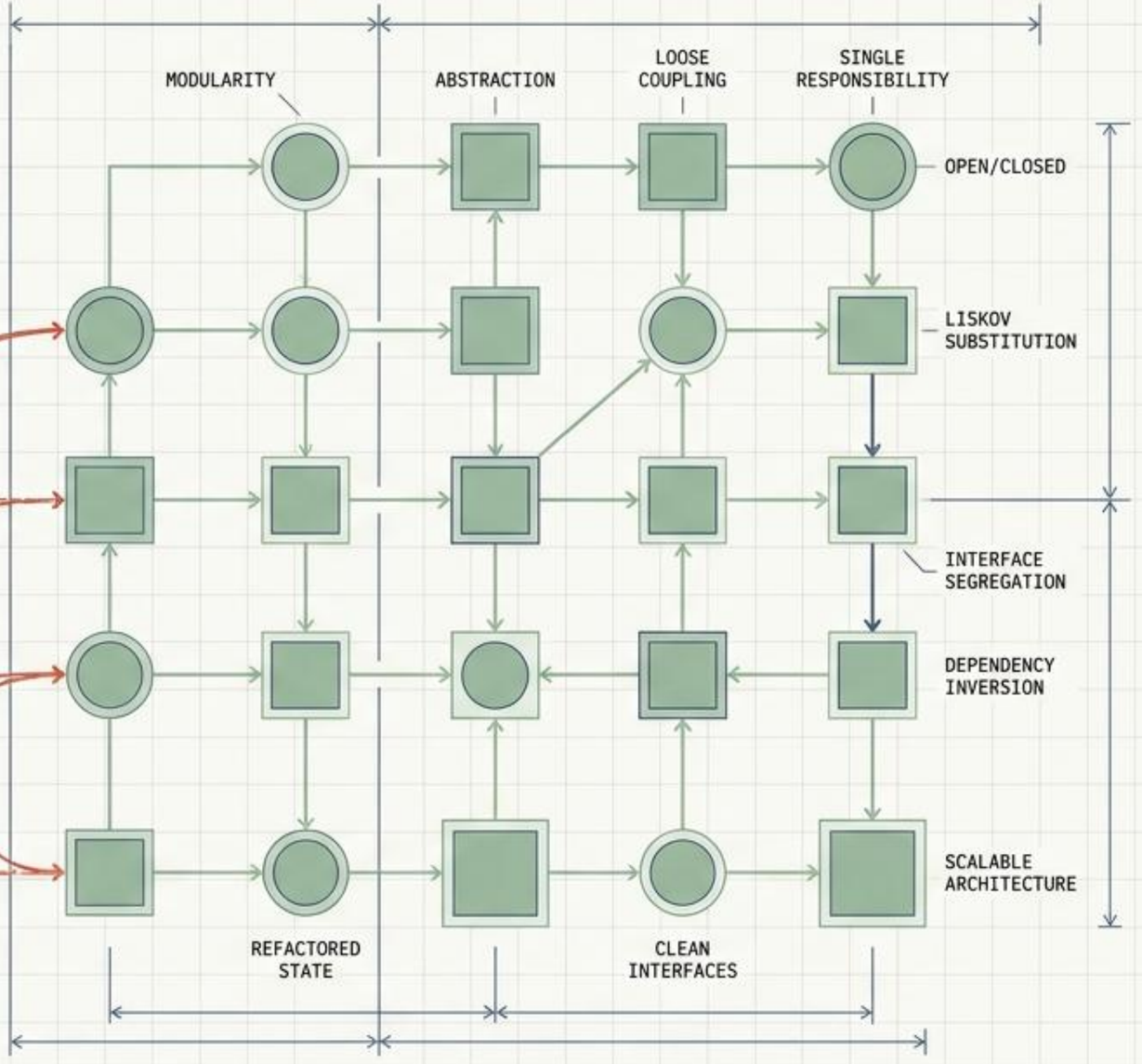


With the physics of object-oriented architecture mastered, we no longer reinvent solutions. We reach for the catalog. The 23 Design Patterns await.

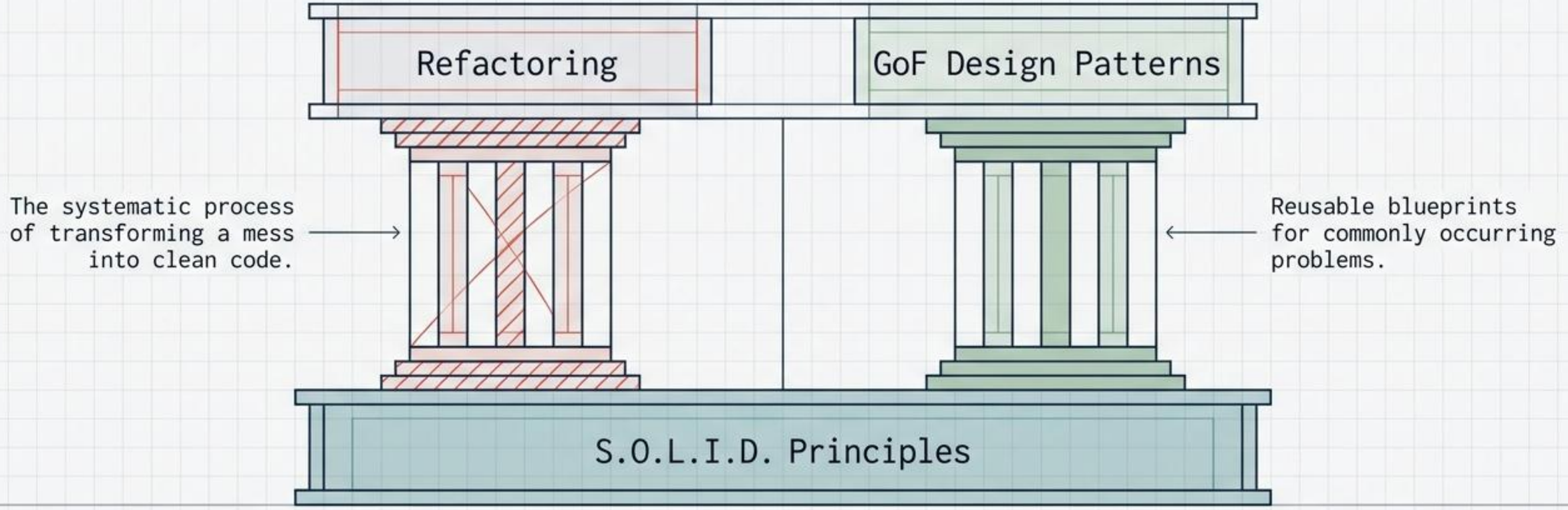
The Architecture of Clean Code

A visual guide to S.O.L.I.D., Refactoring, and Design Patterns.



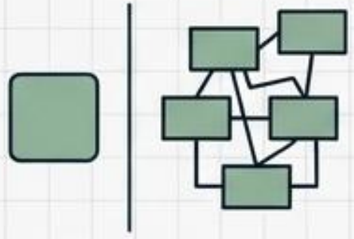
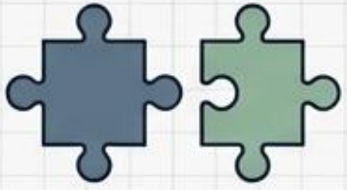
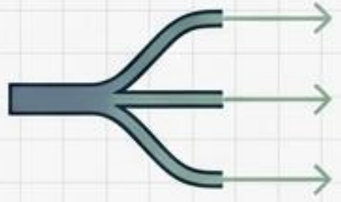
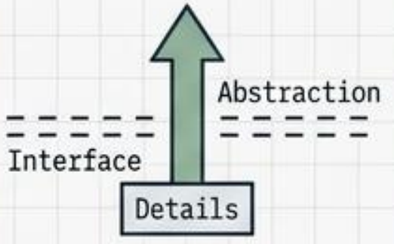


The missing link in software design.

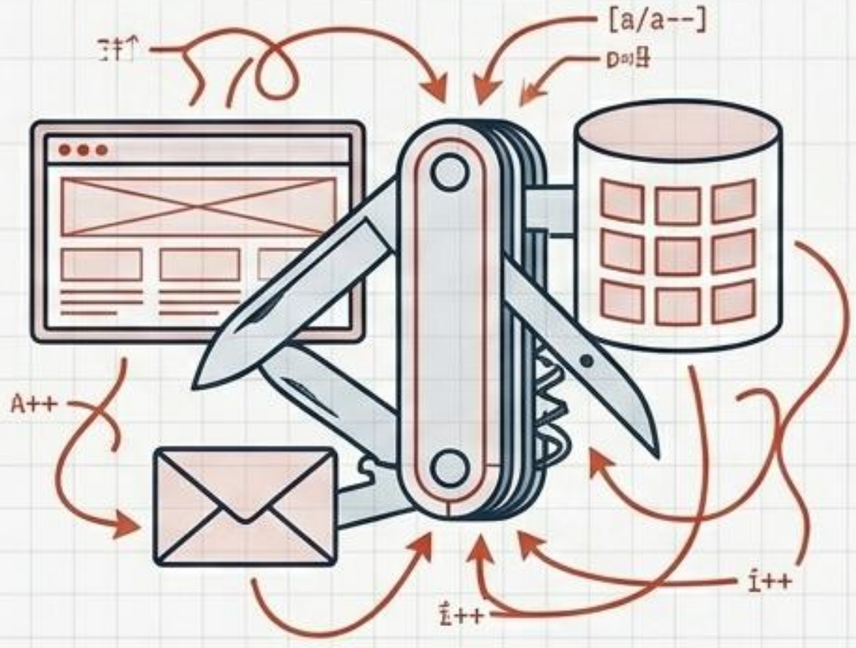


“The connection between refactoring, patterns and general programming principles still remains a mystery for the majority of programmers.” — Alexander Shvets, Refactoring.Guru

S.O.L.I.D. is the foundational language of clean architecture.

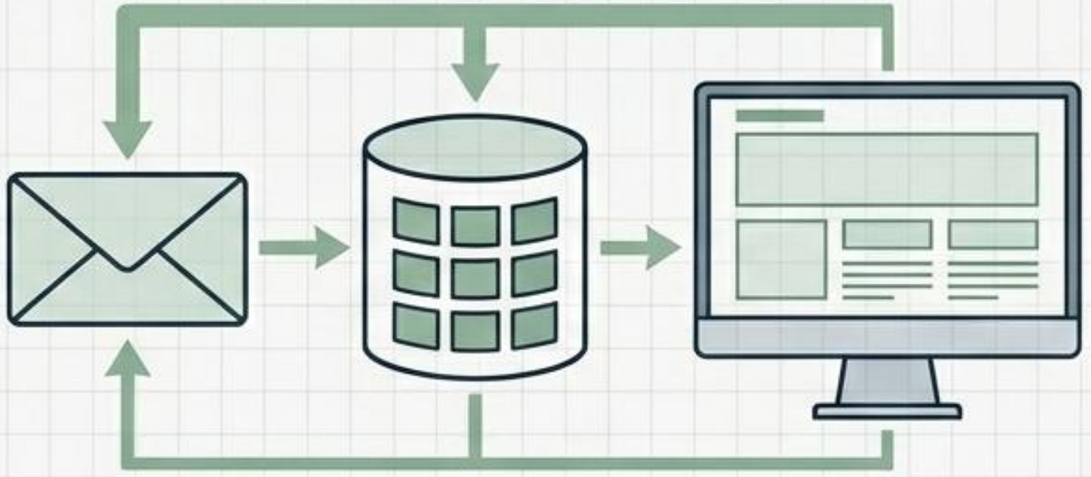
S	O	L	I	D
Single Responsibility	Open/Closed	Liskov Substitution	Interface Segregation	Dependency Inversion
				
One reason to change.	Extend, don't modify.	Interchangeable parts.	Targeted contracts.	Rely on abstractions.

Single Responsibility: A class should have one, and only one, reason to change.



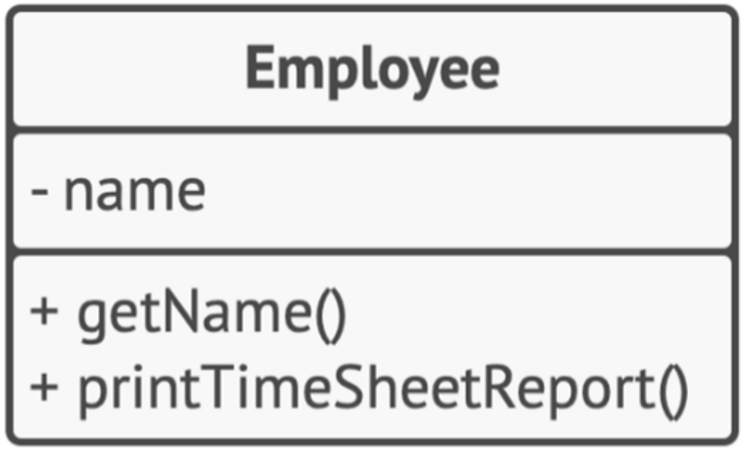
The Problem: Divergent Change.

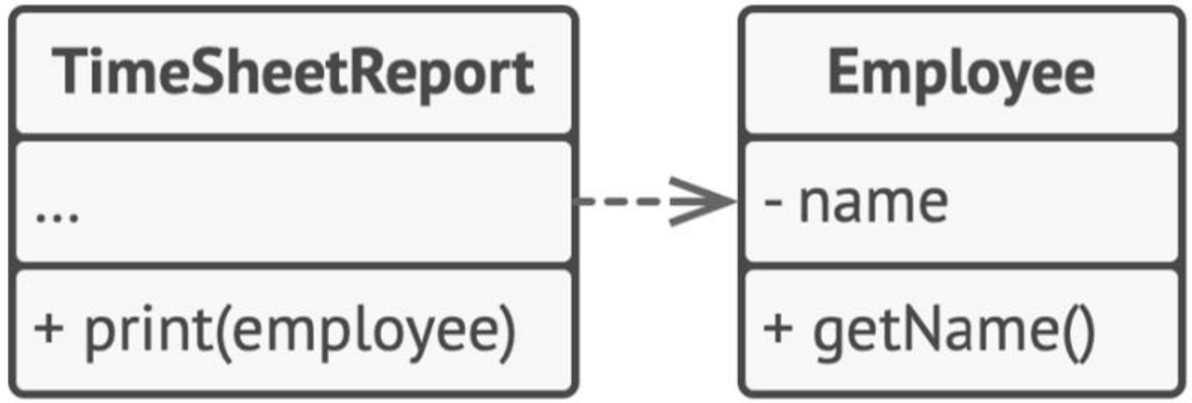
A single class handles multiple distinct domain concepts. When the database schema changes, the UI logic risks breaking.



The Solution: Extract Class.

Delegate distinct responsibilities to highly specialized, isolated objects.

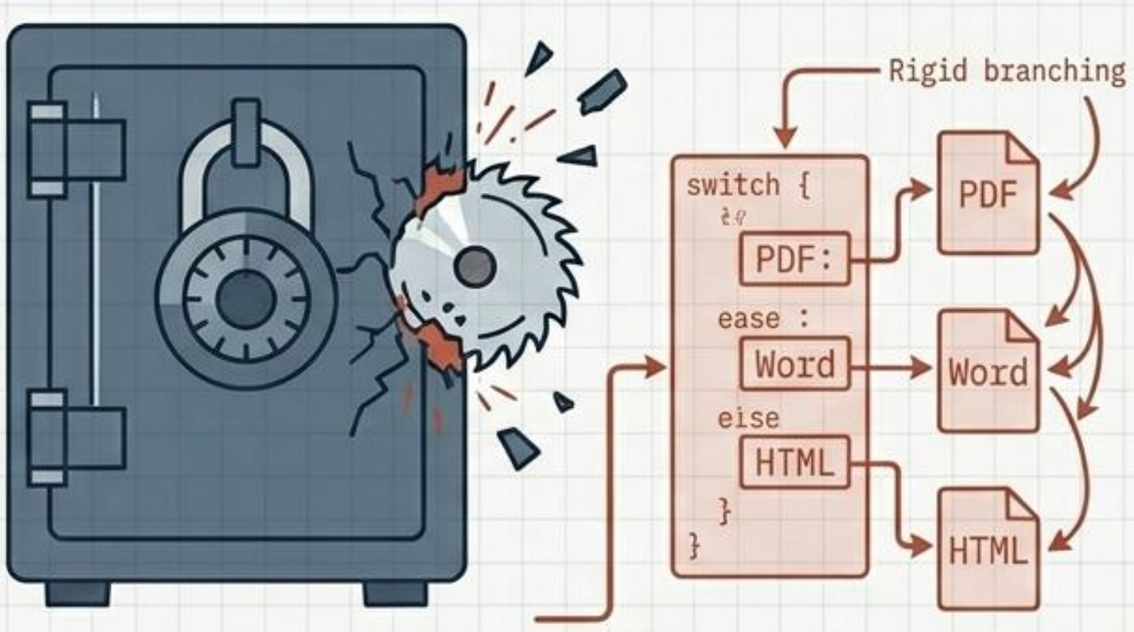





```python
class Report:
    def generate_report(self):
        print("Generating report...")

    def send_email(self):
        print("Sending email...")
```

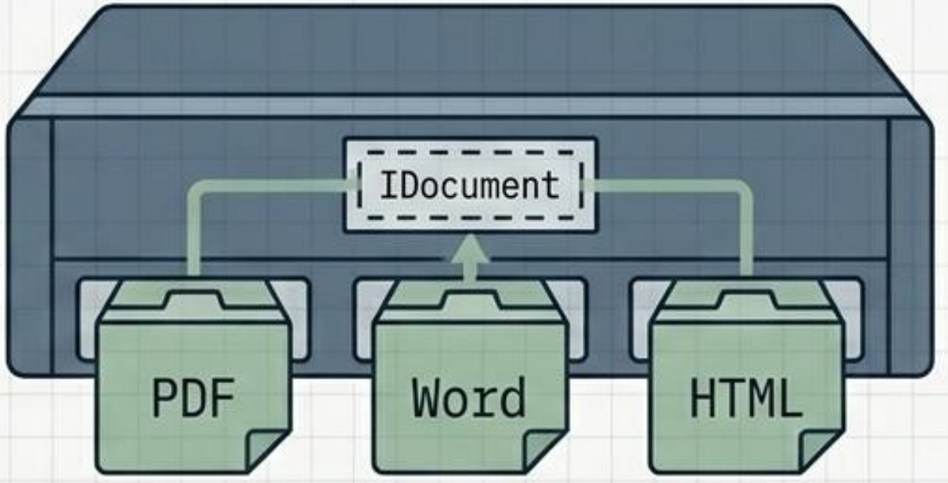
```python
class ReportGenerator:
    def generate_report(self):
        print("Generating report...")

class EmailSender:
    def send_email(self):
        print("Sending email...")
```

Open/Closed: Software entities should be open for extension, but closed for modification.



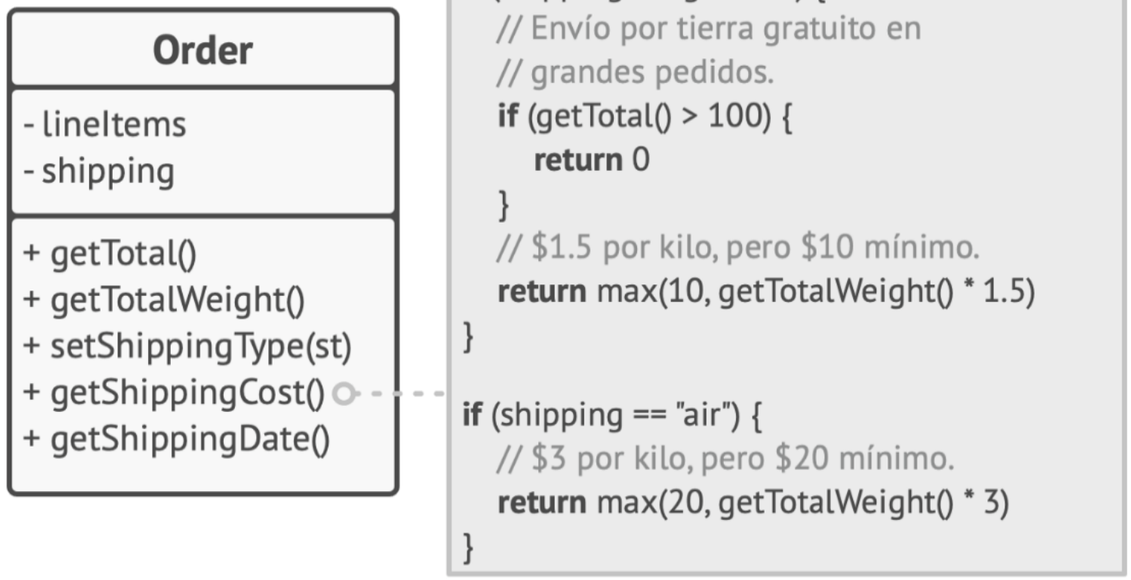
The Problem: Shotgun Surgery

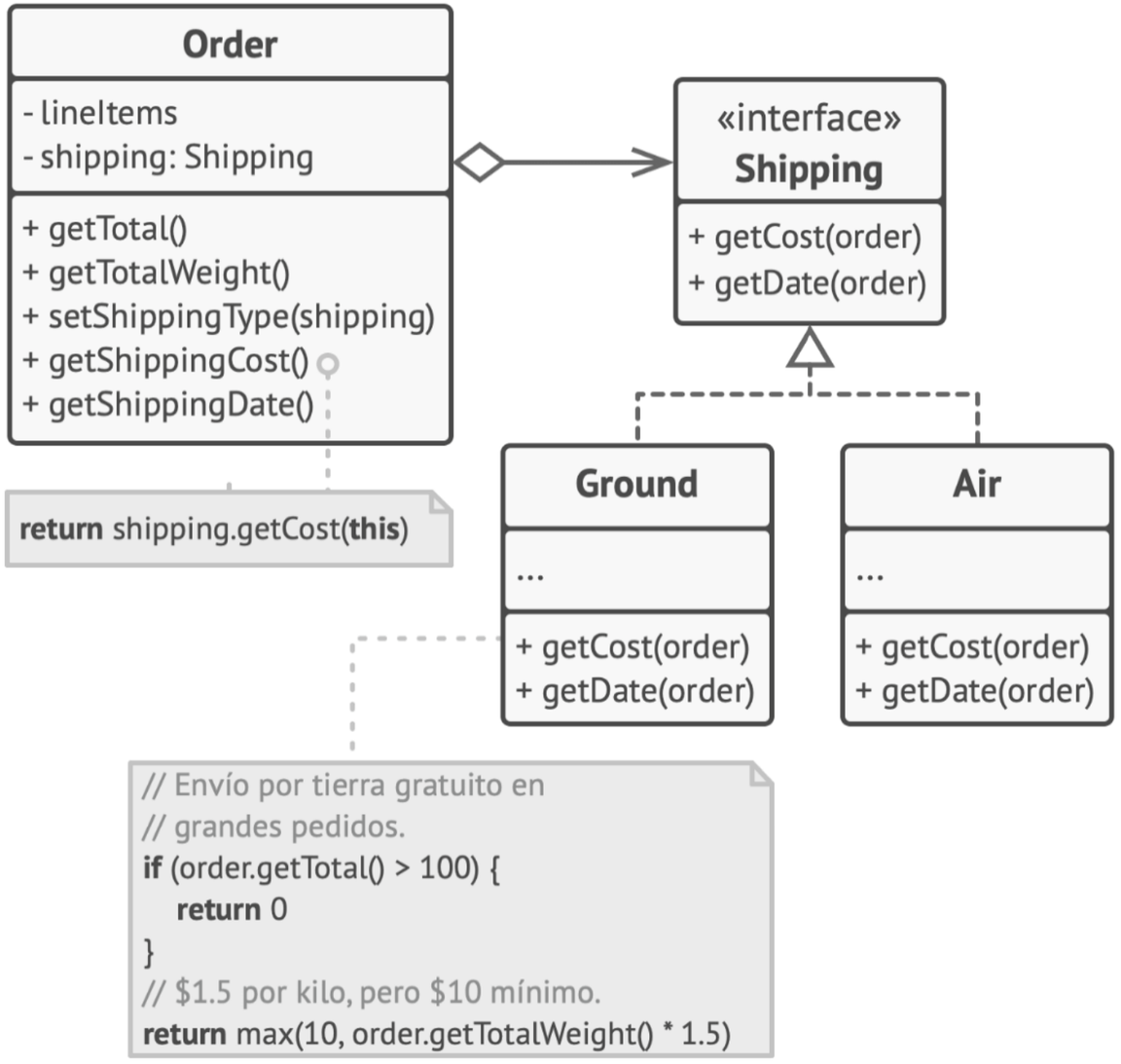
Adding a single new feature requires invasive modifications to existing, tested core logic across multiple files.



The Solution: Replace Conditional with Polymorphism

Use abstract interfaces so new functionality is achieved by adding new code, never altering the old code.

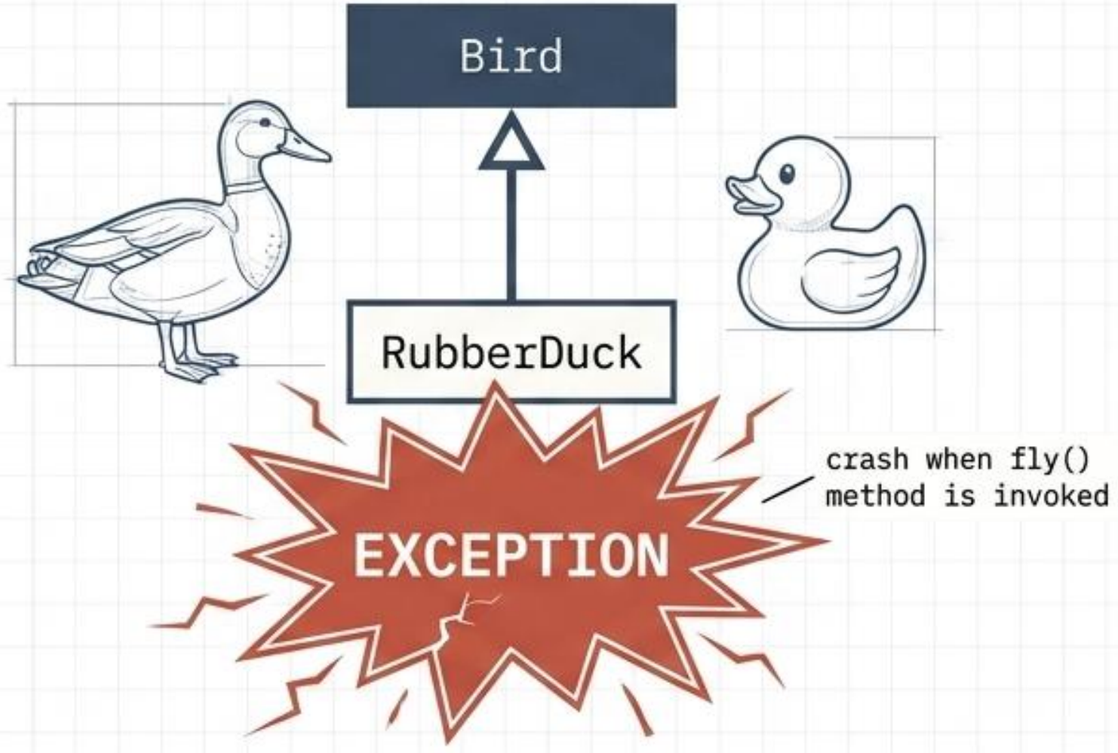

Order
- lineItems
- shipping
+ getTotal()
+ getTotalWeight()
+ setShippingType(st)
+ getShippingCost()
+ getShippingDate()
if (shipping == "ground") {
// Envío por tierra gratuito en
// grandes pedidos.
if (getTotal() > 100) {
return 0
}
// $1.5 por kilo, pero $10 mínimo.
return max(10, getTotalWeight() * 1.5)
}
if (shipping == "air") {
// $3 por kilo, pero $20 mínimo.
return max(20, getTotalWeight() * 3)
}


Order
- lineItems
- shipping: Shipping
+ getTotal()
+ getTotalWeight()
+ setShippingType(shipping)
+ getShippingCost()
+ getShippingDate()
«interface»
Shipping
+ getCost(order)
+ getDate(order)
return shipping.getCost(this)
Ground
...
+ getCost(order)
+ getDate(order)
Air
...
+ getCost(order)
+ getDate(order)
// Envío por tierra gratuito en
// grandes pedidos.
if (order.getTotal() > 100) {
return 0
}
// $1.5 por kilo, pero $10 mínimo.
return max(10, order.getTotalWeight() * 1.5)

```python
class Discount:  2 usages
    def calculate(self, amount, discount_type):
        if(discount_type == "standard"):
            return amount * 0.9
        elif(discount_type == "vip"):
            return amount * 0.8
```

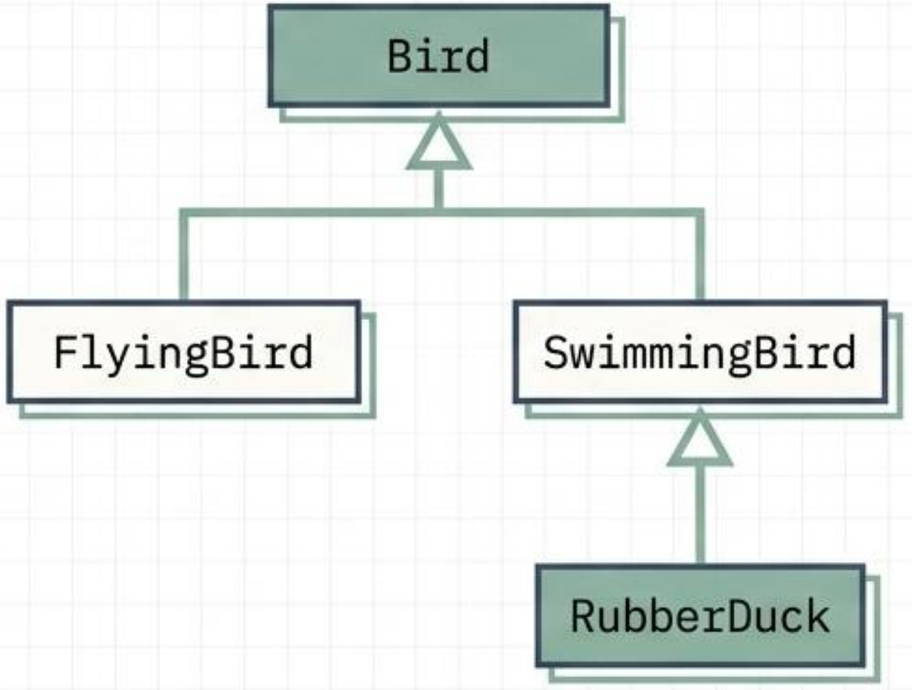
```python
class Discount_Good:
    def calculate(self, amount):
        return amount

class StanderDiscount(Discount):

    def calculate(self, amount):
        return amount * 0.9

class VIPDiscount(Discount):
    def calculate(self, amount):
        return amount * 0.8
```

Liskov Substitution: Subtypes must be completely substitutable for their base types.



The Problem: Refused Bequest

A subclass inherits methods it cannot functionally use, violating the behavioral contract of the parent class and crashing the system.



The Solution: Extract Superclass

Redesign the hierarchy to ensure subclasses honor the expected behavior of the abstraction they claim to represent.

Liskov Substitution Principle

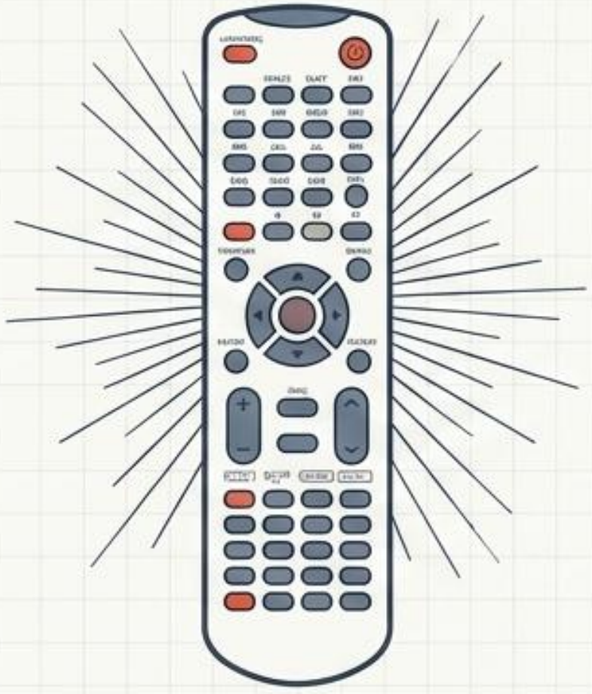
- Parameter types in the subclass method must match the parameter types of the superclass method.
- The return type in the subclass method must match the return type of the superclass method.
- A subclass method must not throw [new or broader] exception types that are expected to be thrown by the base method.
- A subclass must not strengthen preconditions.
- A subclass must weaken postconditions.
- A subclass must not change the values of private fields of the superclass

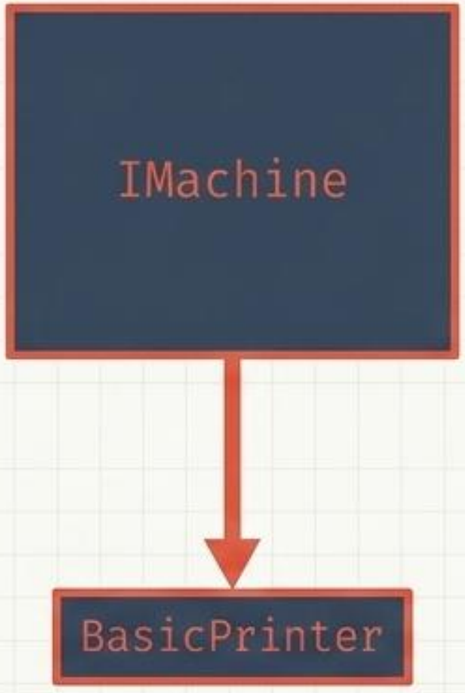
```python
class Bird:  1 usage
    def fly(self):
        print("Flying...")

class Penguin(Bird):
    def fly(self):
        raise Exception("I can't fly!")
```

```python
class Bird:  2 usages
    def move(self):
        print("Moving...")

class FlyingBird(Bird):
    def move(self):
        print("Flying...")

class Penguin(Bird):
    def move(self):
        print("Swimming...")
```

Interface Segregation: *Clients should not be forced to depend on methods they do not use.*

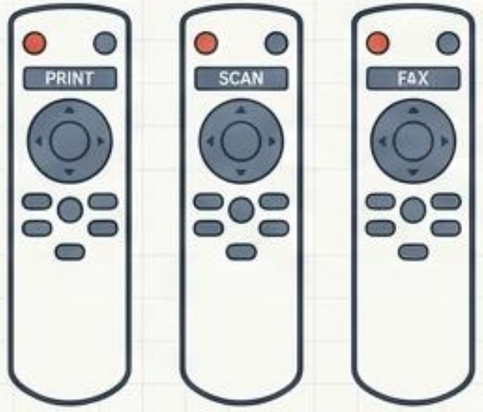
The Problem: Large Class / Bloaters

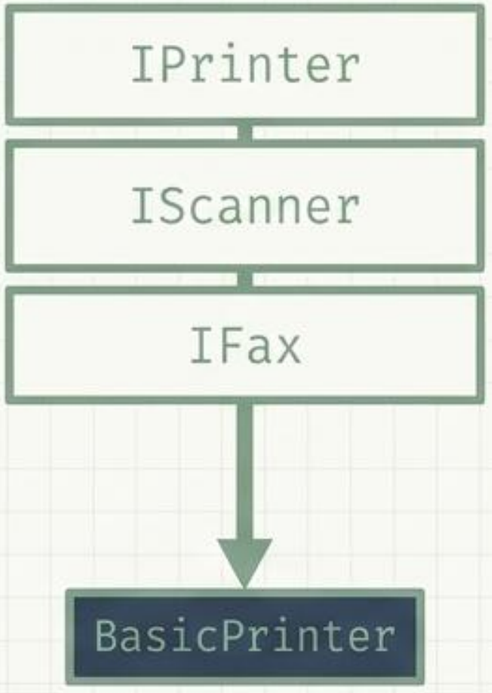




Changes to an unused method force unnecessary recompilation and ripple effects in entirely unrelated client classes.

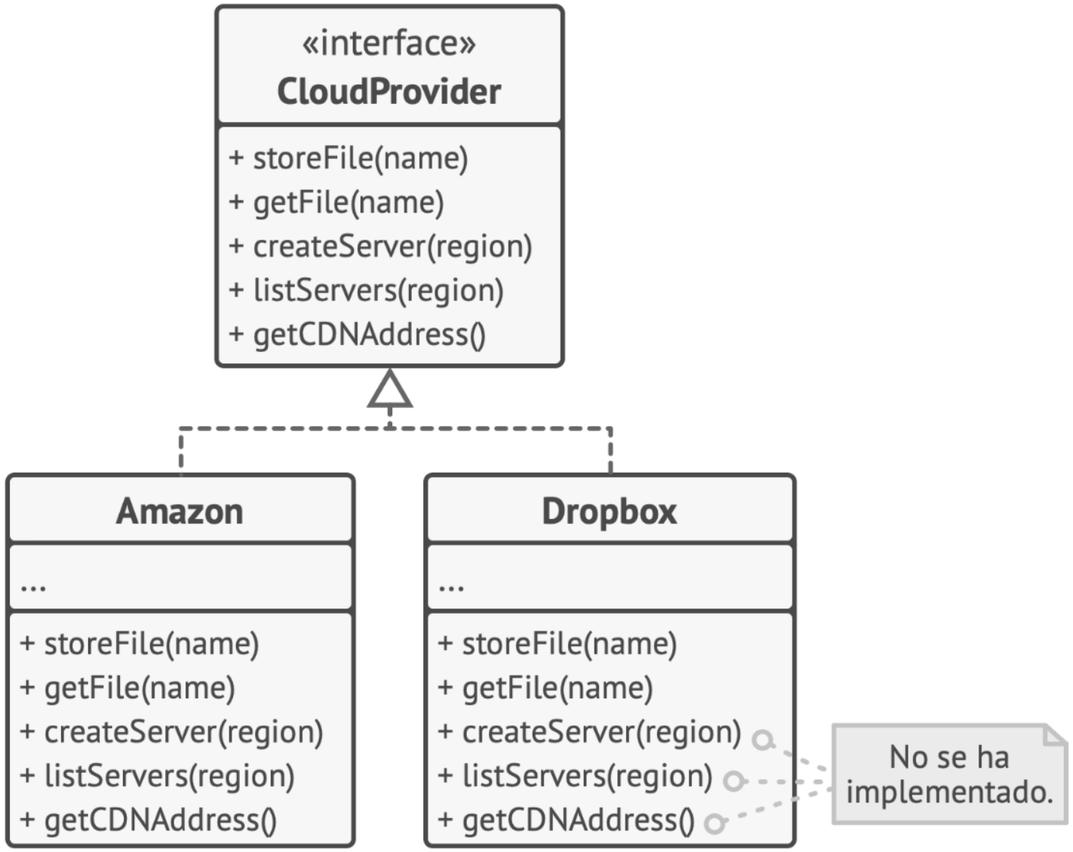
The Solution: Extract Interface

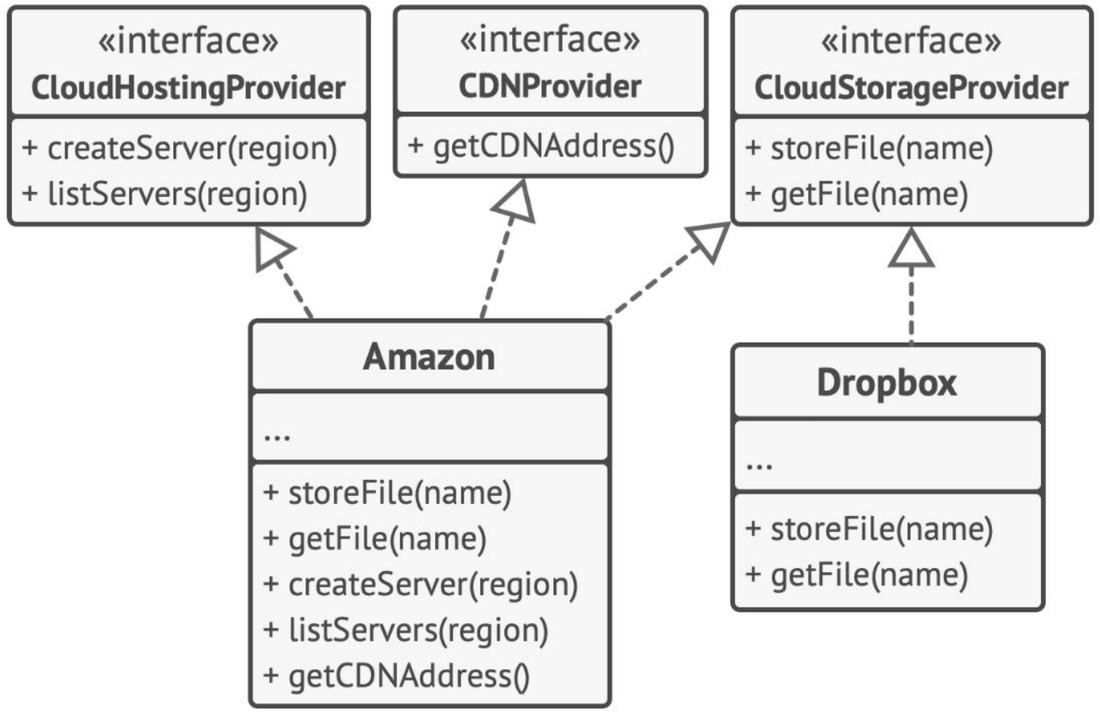




Divide fat interfaces into cohesive, highly specific, client-focused contracts.

Interface Segregation Principle



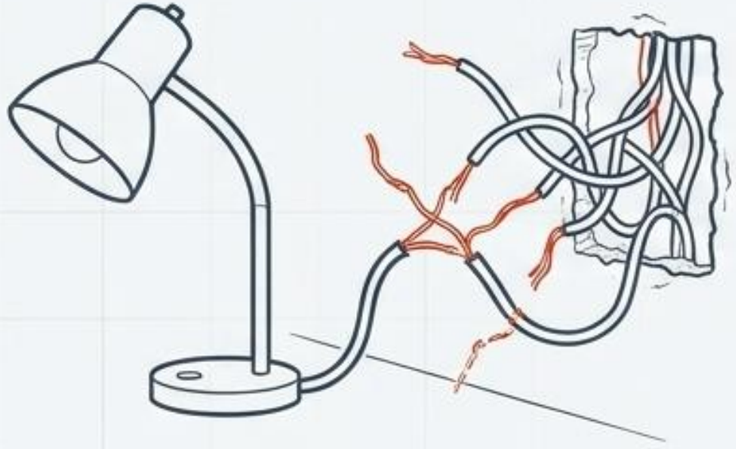


```python
class Worker:  1 usage
    def work(self):
        pass

    def eat(self):
        pass

class Developer(Worker):
    def work(self):
        print("Writing code...")

    def eat(self):
        raise NotImplementedError("I don't need this method!")
```

```python
class Workable:  2 usages
    def work(self):
        pass

class Eatable:  1 usage
    def eat(self):
        pass

class Developer(Workable):
    def work(self):
        print("Writing code...")

class Chef(Workable, Eatable):
    def work(self):
        print("Cooking...")

    def eat(self):
        print("Eating food...")
```

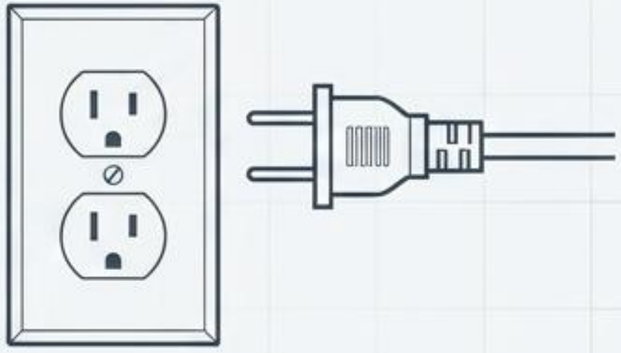
Dependency Inversion: High-level modules and low-level details must depend on abstractions.

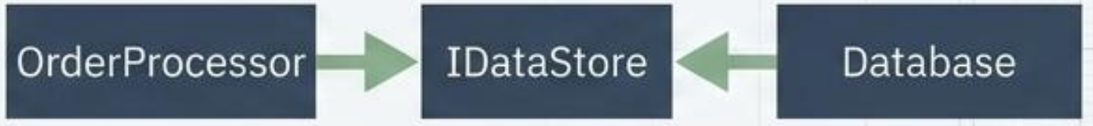
The Problem: Inappropriate Intimacy



Business logic is **rigidly hardwired** to **specific infrastructure details**, making testing impossible and swapping technologies a massive risk.

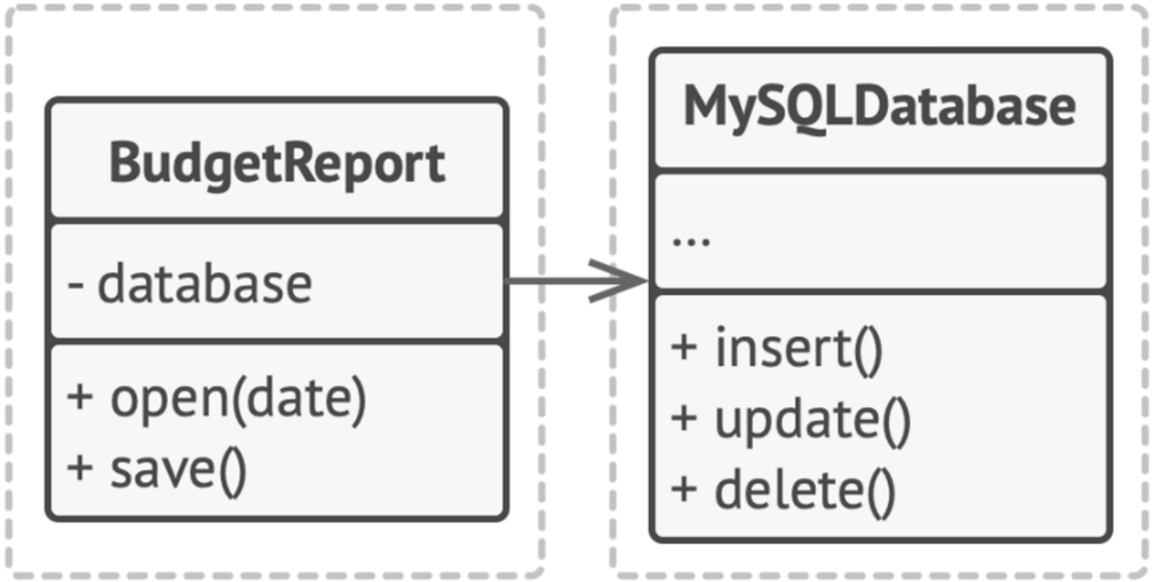
The Solution: Hide Delegate / Extract Interface

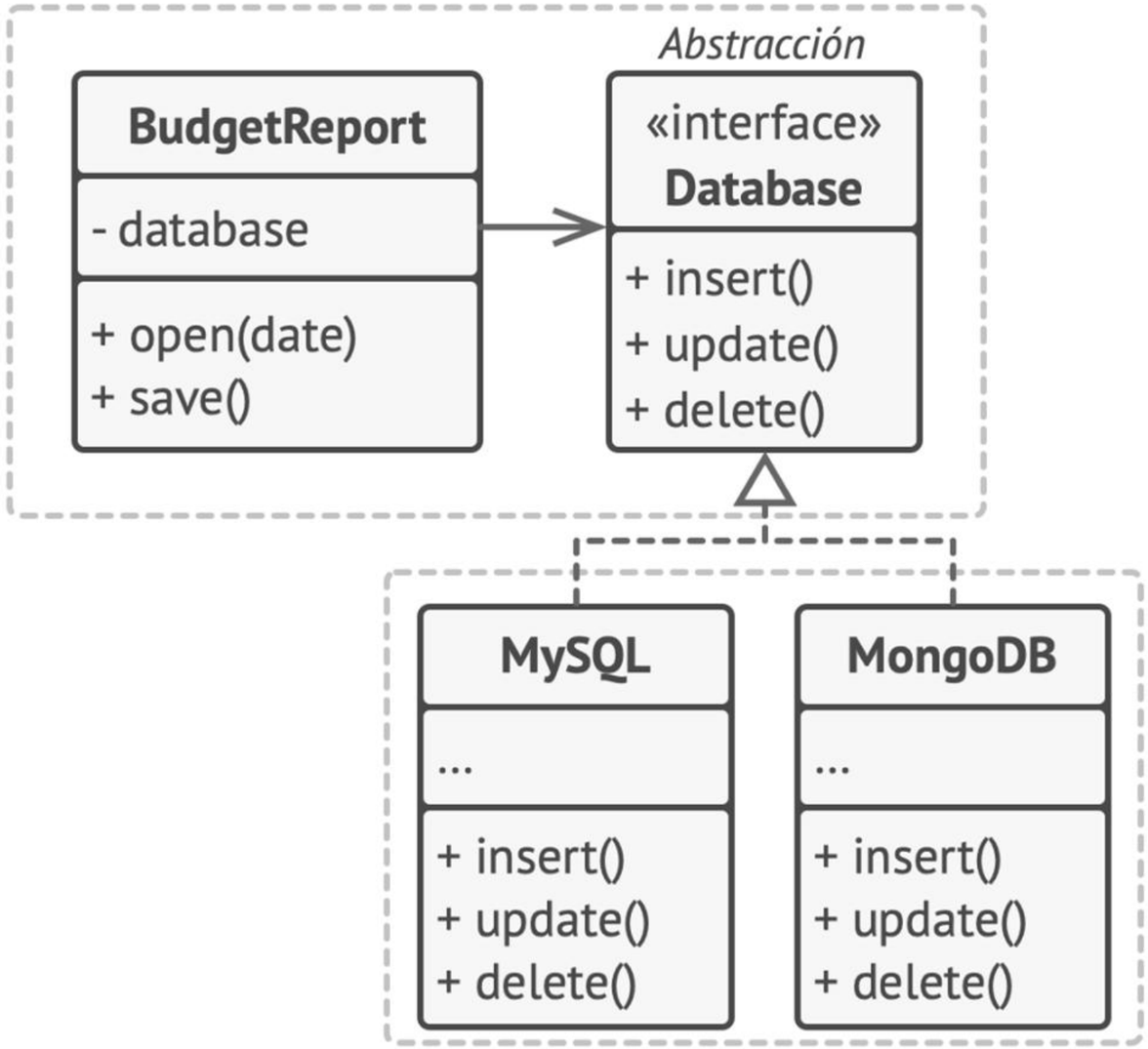




Introduce an abstraction layer. The lamp and the power grid never know about each other; they only agree on the socket.

Dependency Inversion Principle





```python
class BackendDeveloper:  1 usage
    def develop(self):  1 usage
        print("Writing backend code...")

class Project:
    def __init__(self):
        self.developer = BackendDeveloper()

    def develop_project(self):
        self.developer.develop()
```

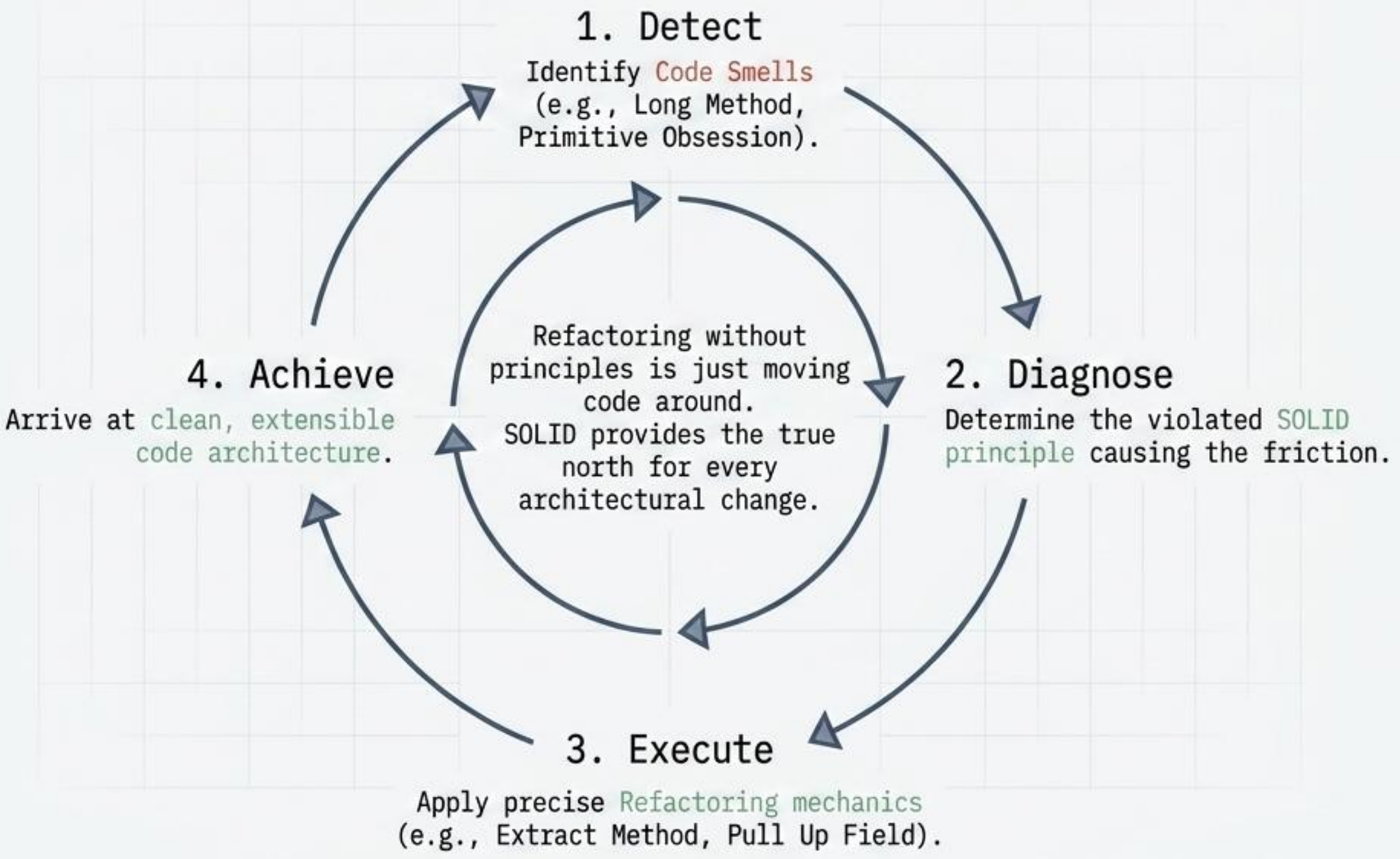
```python
class Developer:  3 usages
    def develop(self):  1 usage
        pass

class BackendDeveloper(Developer):  2 usages
    def develop(self):  1 usage
        print("Writing backend code...")

class FrontendDeveloper(Developer):
    def develop(self):
        print("Writing frontend code...")

class Project:  1 usage
    def __init__(self, developer: Developer):
        self.developer = developer

    def develop_project(self):  1 usage
        self.developer.develop()
```

The Architecture Matrix: Linking smells to patterns.

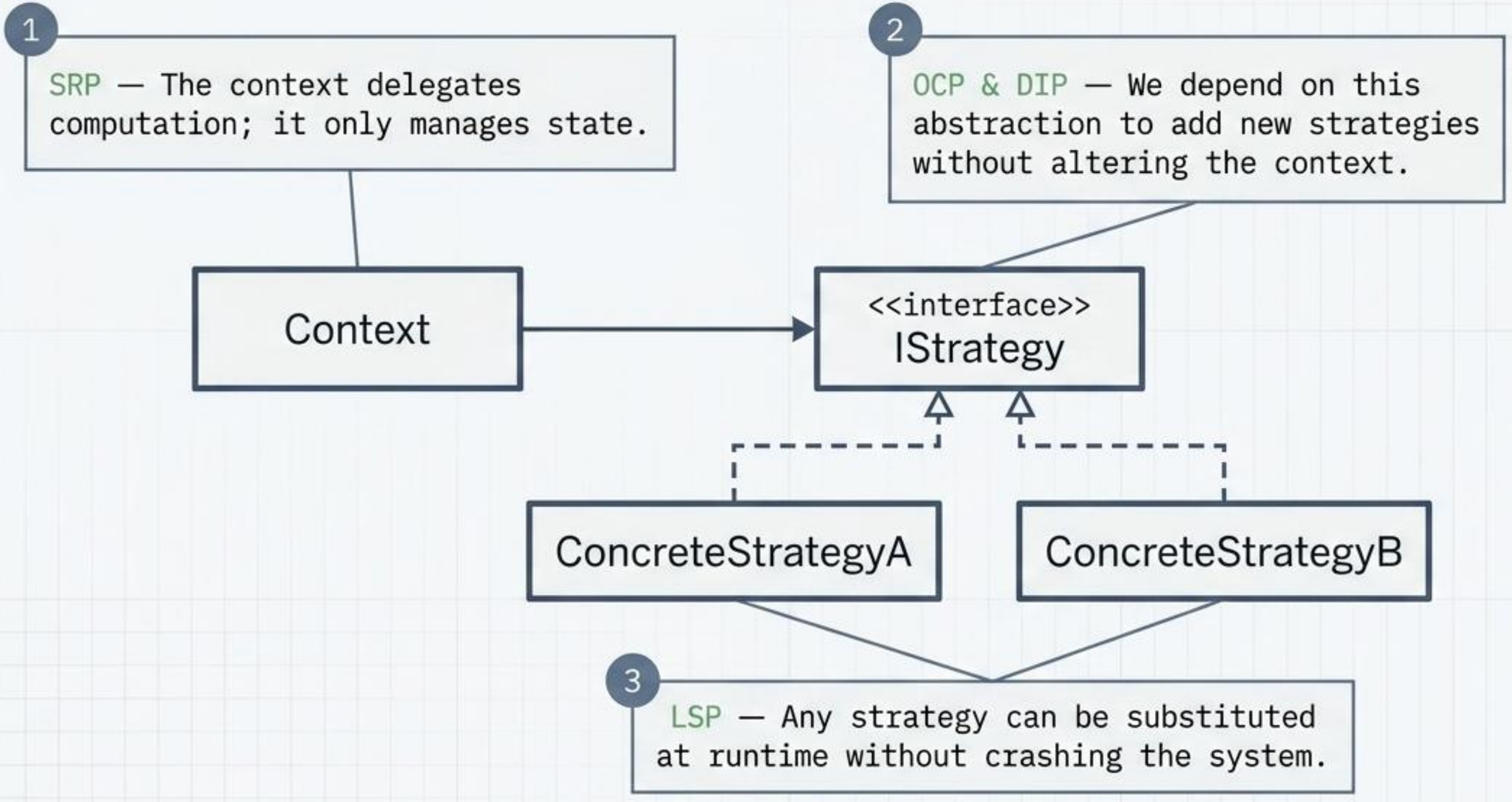
The Diagnostic (Code Smell)	The Missing Link (S.O.L.I.D.)	The Blueprint (GoF Pattern)
Divergent Change	Single Responsibility	Command, Facade
Shotgun Surgery	Open/Closed	Strategy, Decorator
Refused Bequest	Liskov Substitution	Template Method
Fat Interfaces / Large Class	Interface Segregation	Adapter
Inappropriate Intimacy	Dependency Inversion	Abstract Factory, Builder

The principles dictate the required structure; the Refactorings are the physical moves to achieve it; the GoF Patterns are the final resulting shapes.

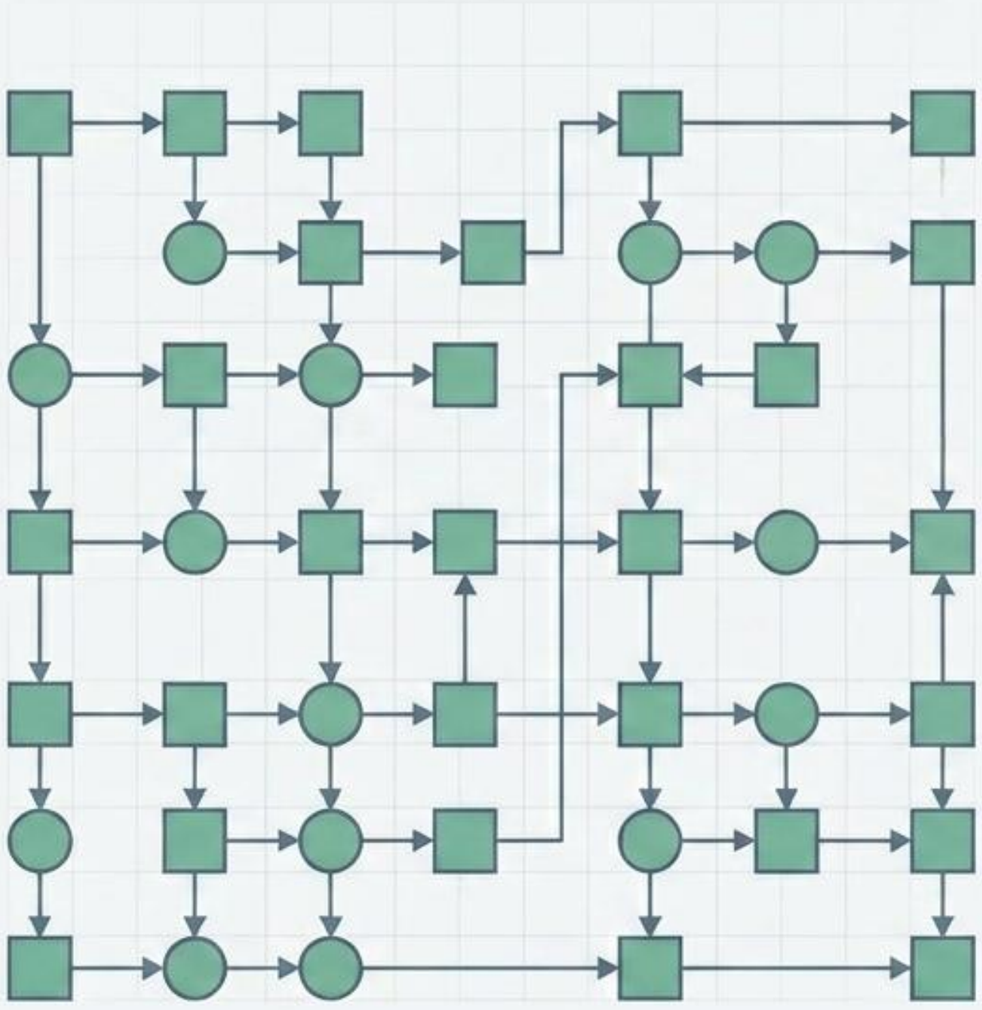
Refactoring is applied SOLIDity.



Design patterns are SOLID in action.



From mess to mastery.



Smells are the symptoms of architectural friction.

S.O.L.I.D. provides the diagnostic framework to understand the flaw.

Refactoring supplies the precise mechanics to fix it.

Design Patterns are the documented, reusable outcomes of that fix.

Based on the foundational methodologies documented in *Design Patterns: Elements of Reusable Object-Oriented Software* and by Alexander Shvets at Refactoring.Guru.